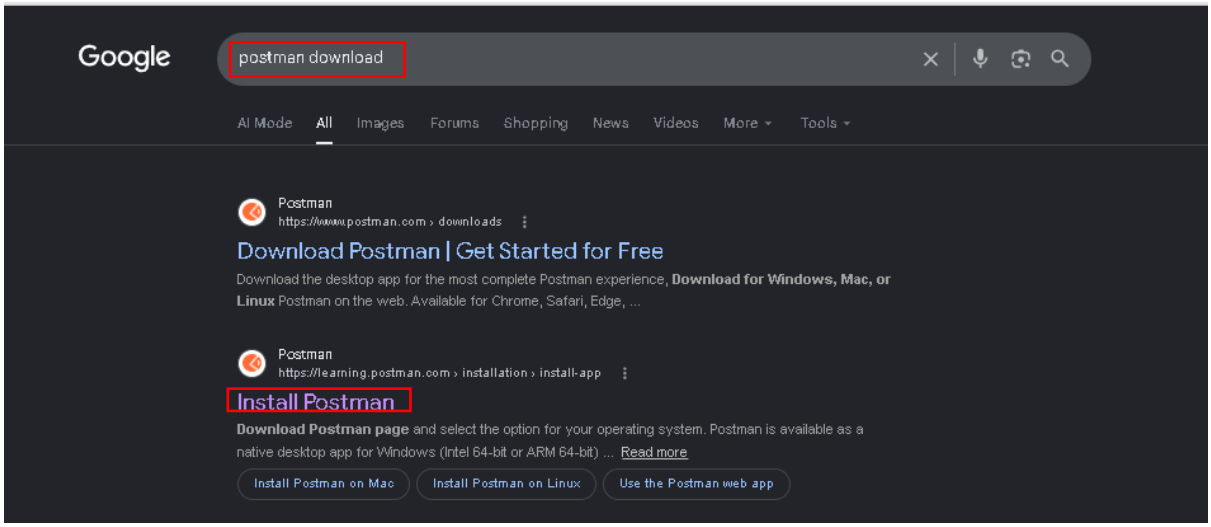
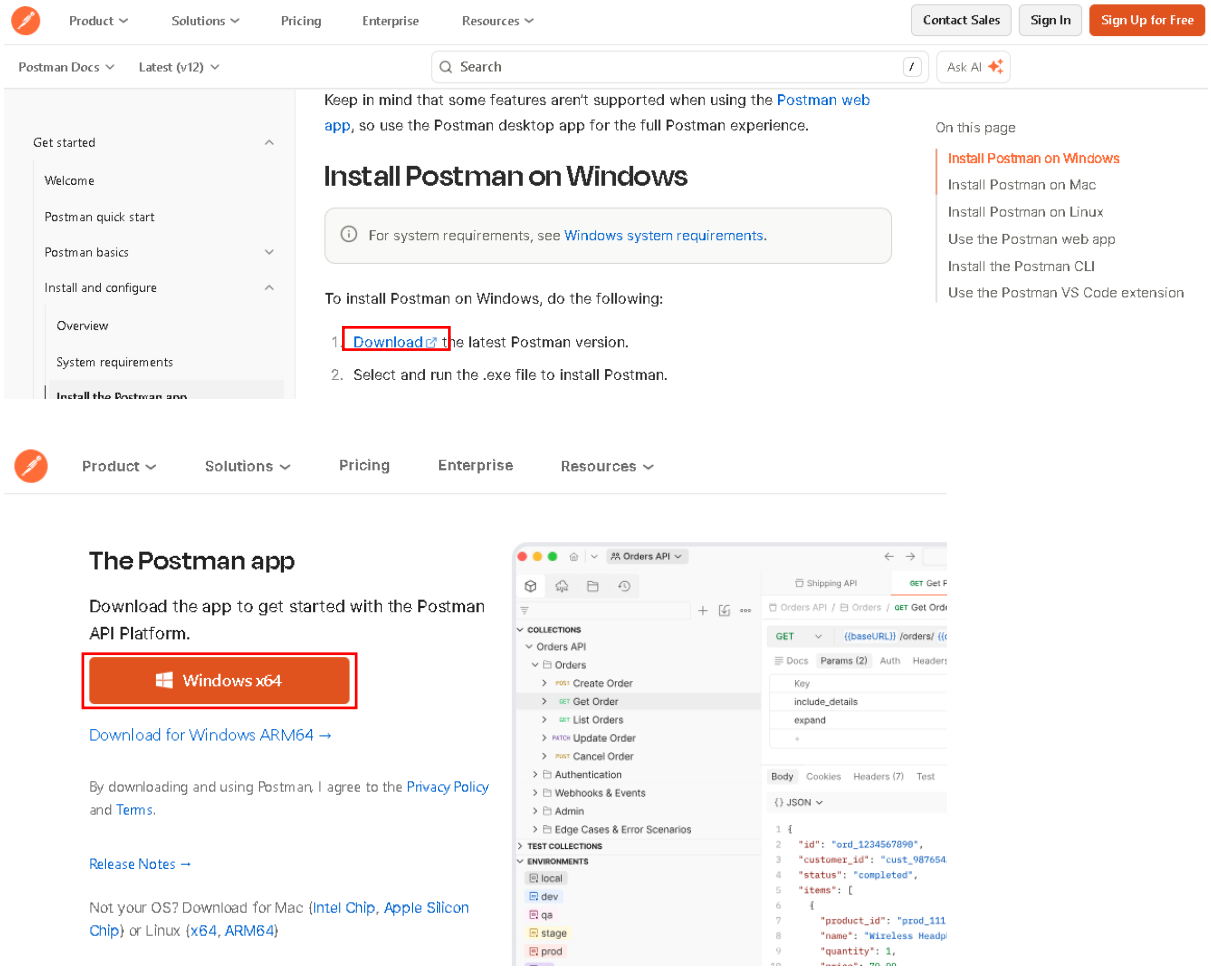
1. How to create RESTful service & Install Postman

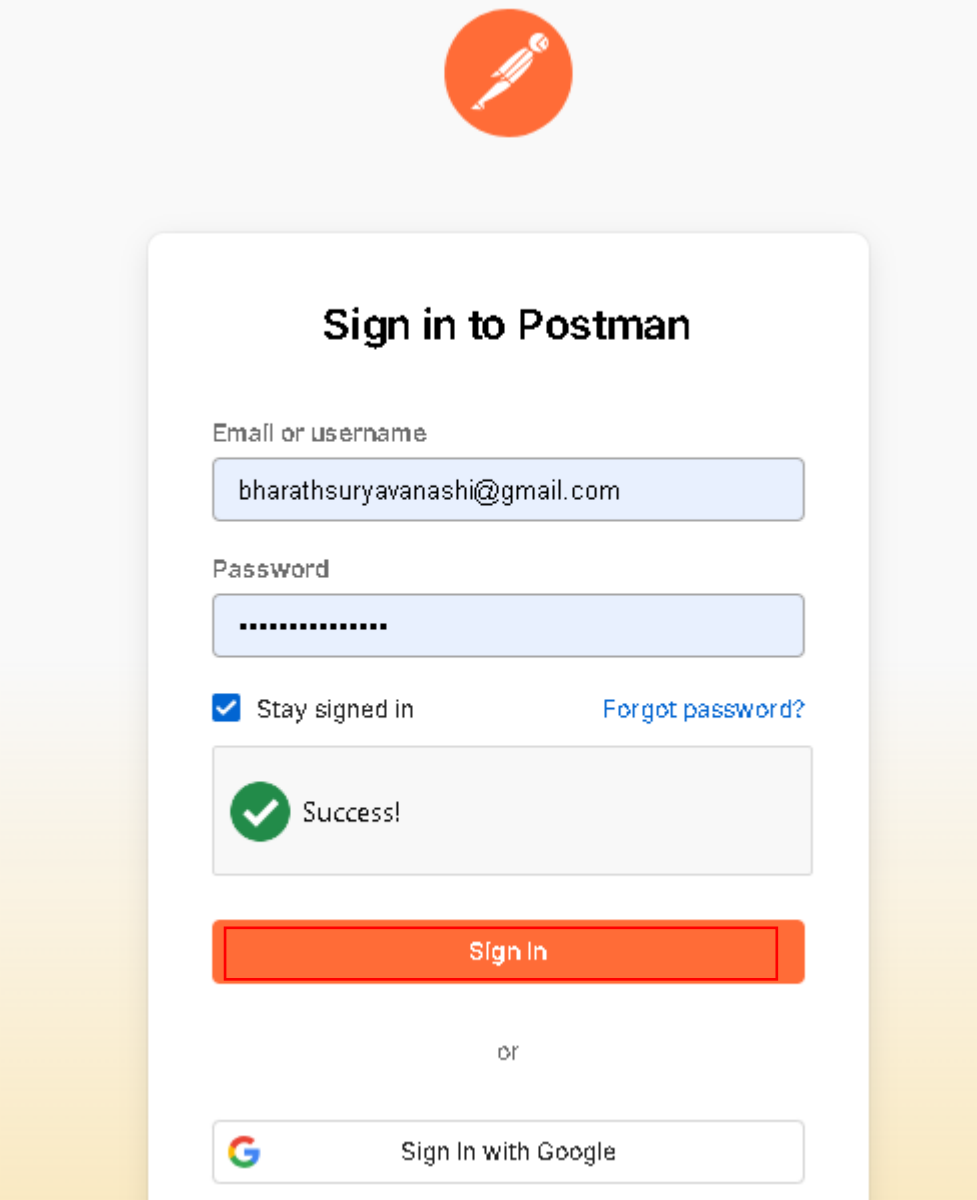
Step 1: Go to Web Browser search for 'Postman' select the second option here.



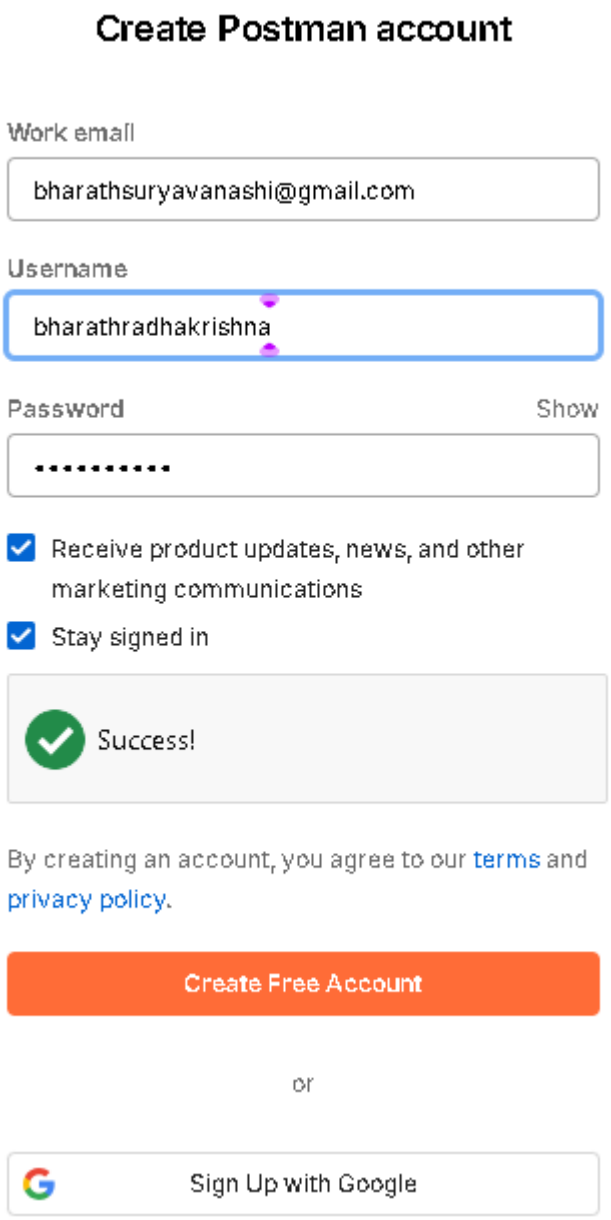
Step 2: Now click on 'Windows 64-bit' option to download for windows OS.



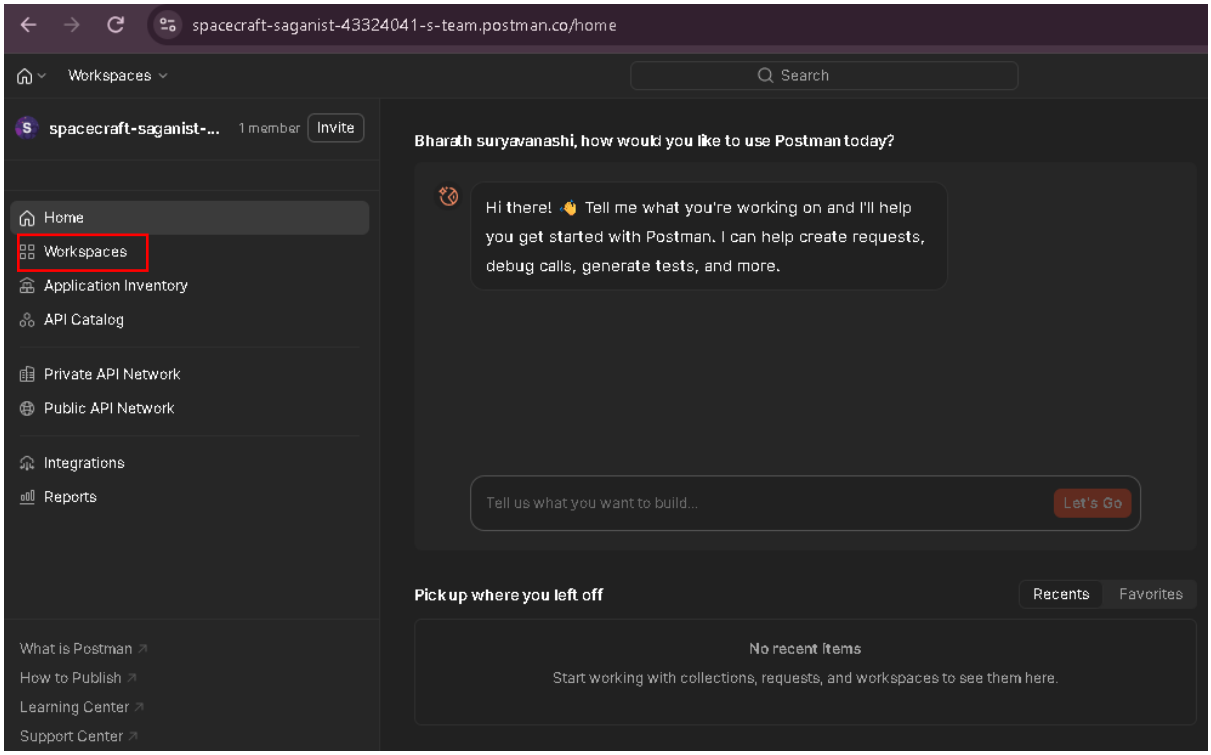
Step 3: You can simply sign to your account by giving your Email-id and Password here.



Step 4: If you don't have an account, click on 'Create Postman Account' by giving your Email-id,Username & Password. Then click on 'Create free account' here.



Step 5: This is your POSTMAN workspace.



Test created APIs with the help of Postman

HTTP Methods:

HTTP GET - Use GET requests to retrieve resource representation/information only – and not modify it in any way. As GET requests do not change the resource's state, these are said to be safe methods.

HTTP POST - Use POST APIs to create new subordinate resources, e.g., a file is subordinate to a

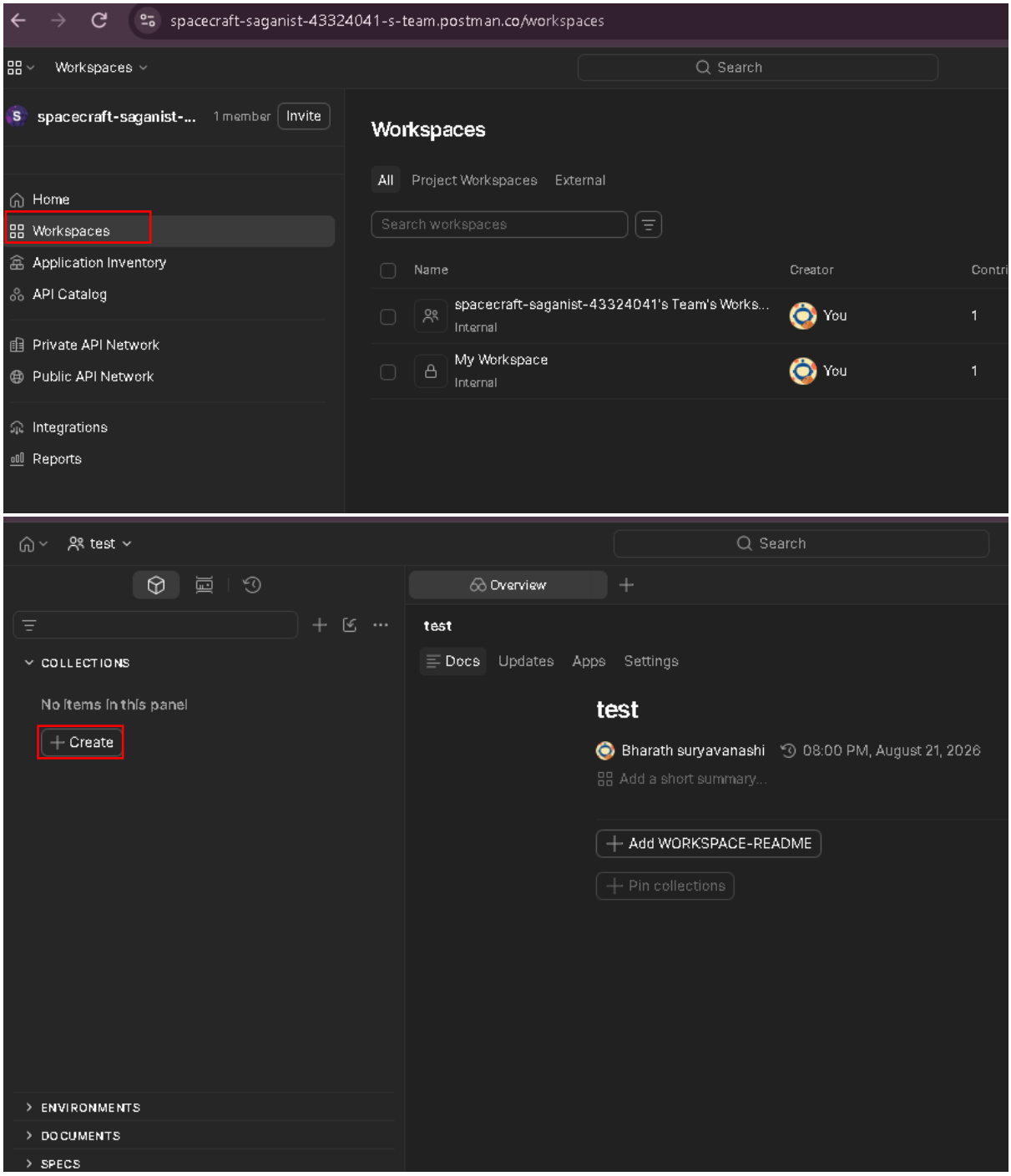
directory containing it or a row is subordinate to a database table.

HTTP PUT - Use PUT APIs primarily to update an existing resource (if the resource does not exist, then API may decide to create a new resource or not).

HTTP DELETE - As the name applies, DELETE APIs delete the resources (identified by the Request-

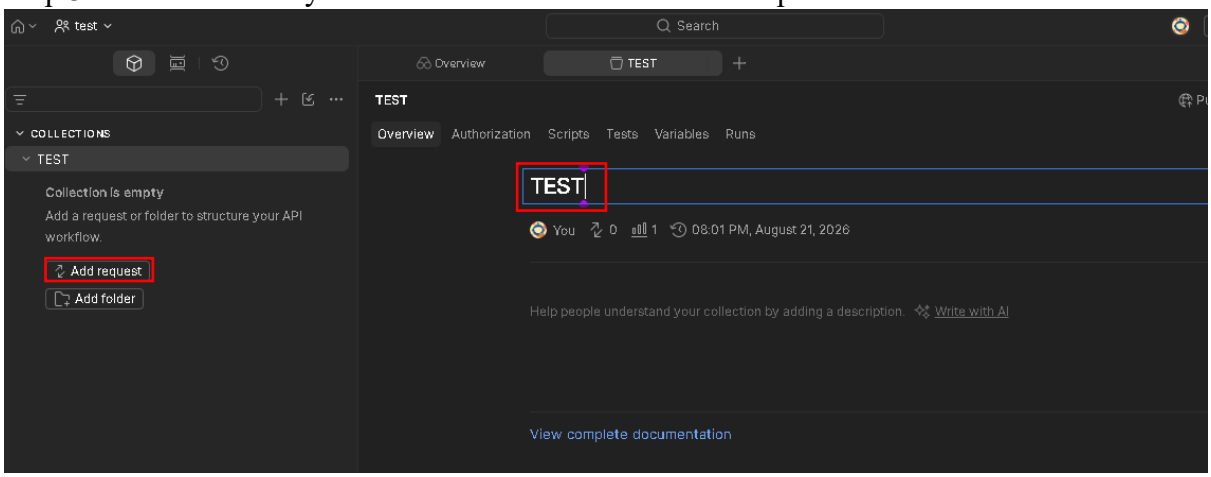
URI). DELETE operations are idempotent. If you DELETE a resource, it's removed from the collection of resources.

Step 1: Now click on 'Create Collection' here.

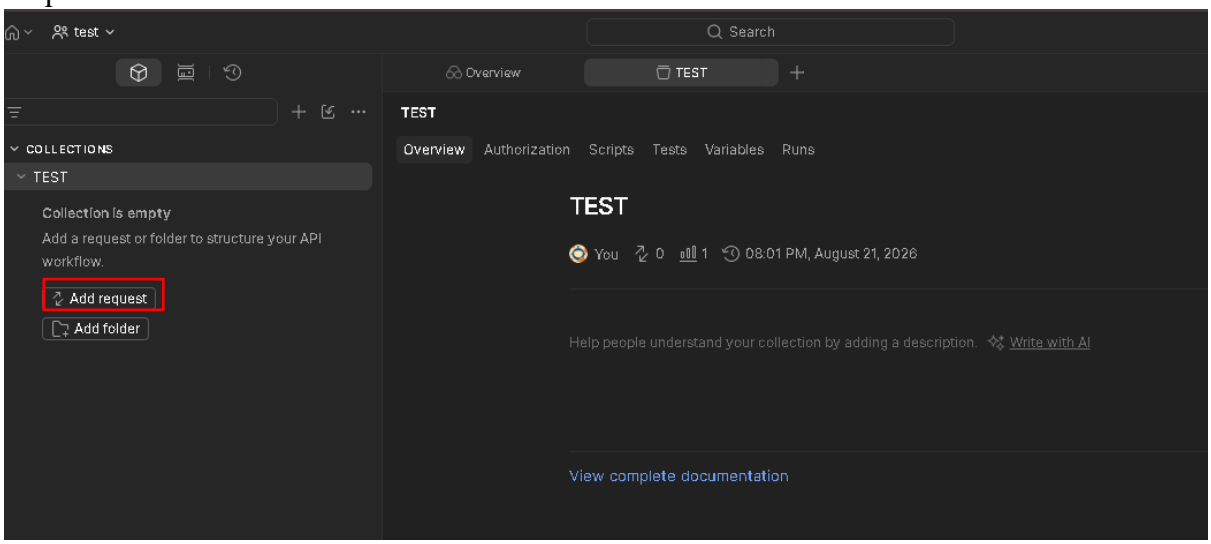


Step 2: Again click on ‘Create a Collection’.

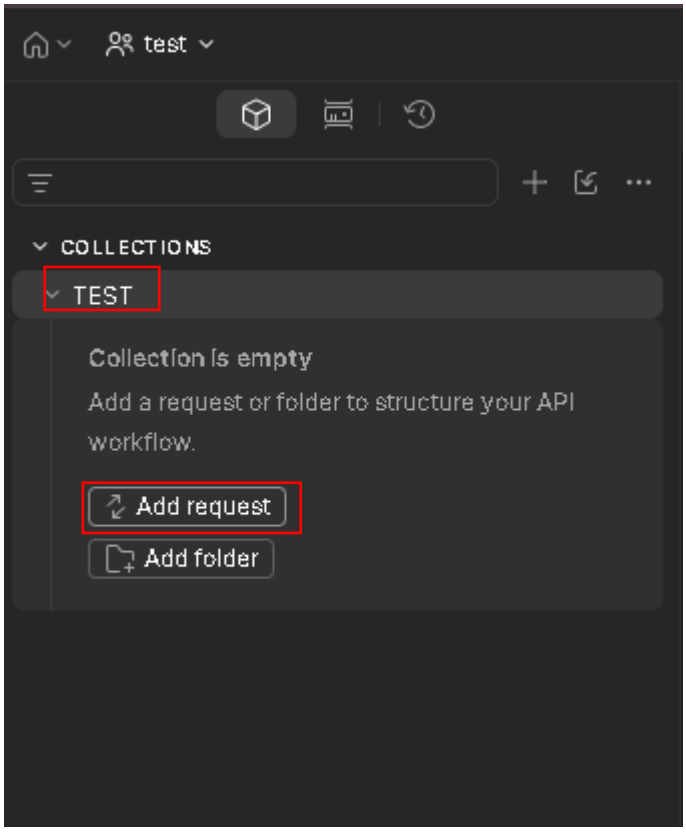
Step 3: Give a name to your collection & click on ‘Create’ option as shown below.



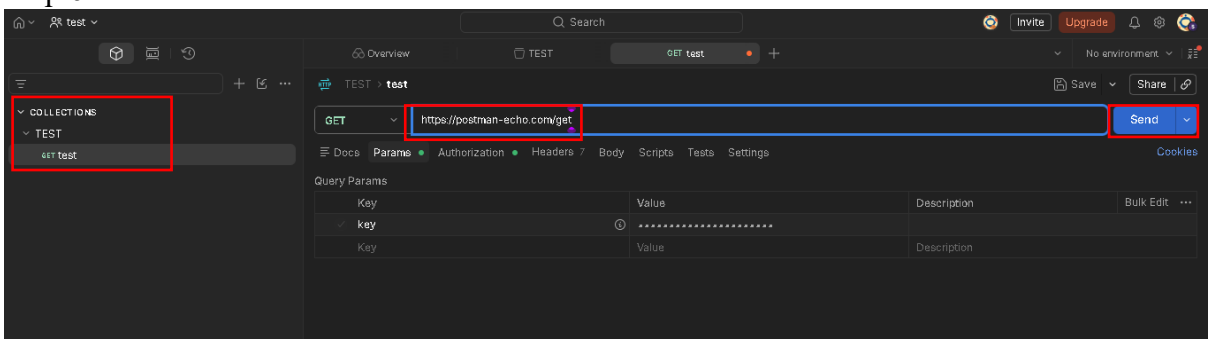
Step 4: Now click on ‘Save’.



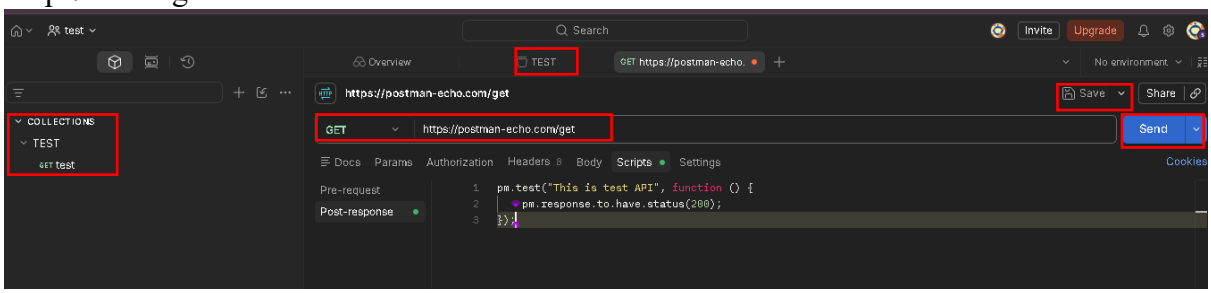
Step 5: Click on ‘add request’ option here.



Step 6: Now select 'GET' method & click on the HTTP link as shown below.

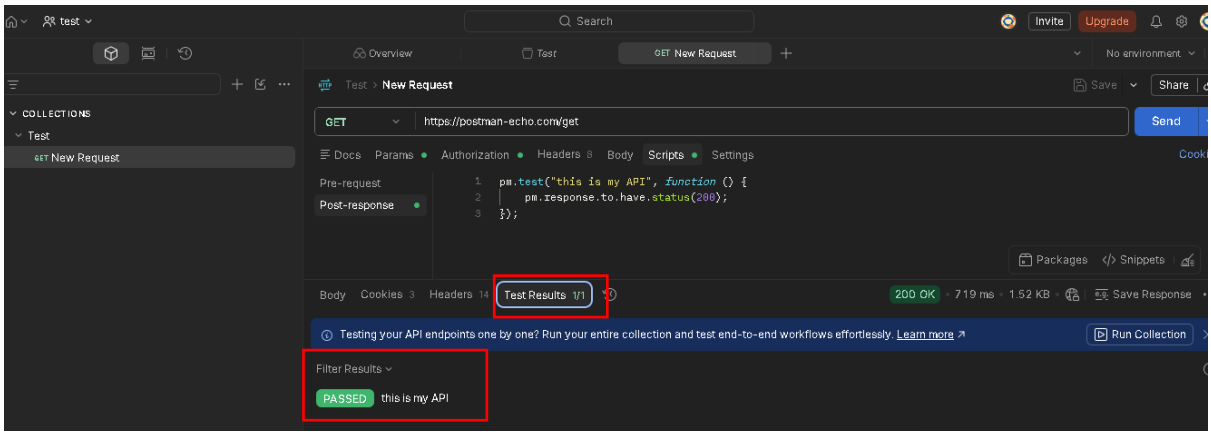


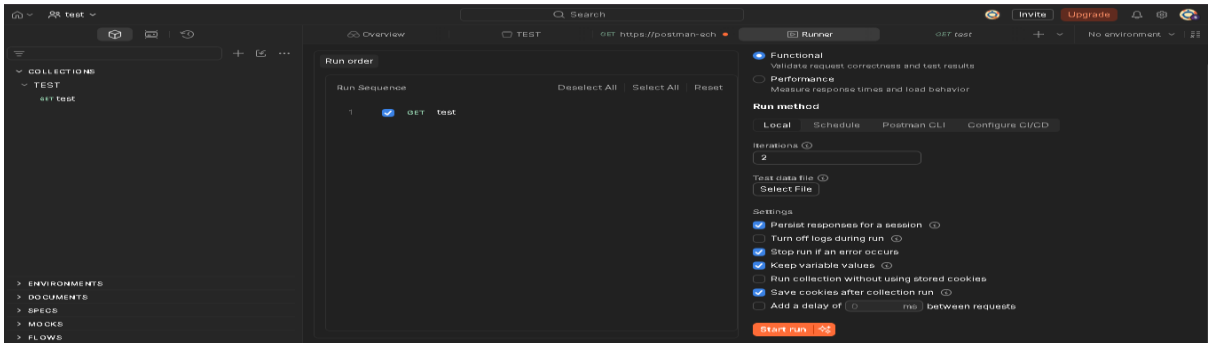
Step 7: Now go to → Tests → Enter the below code → Save it & Send.



Step 8: Then check the response status here. If the status is 200 OK means the response has been accepted by the POSTMAN.

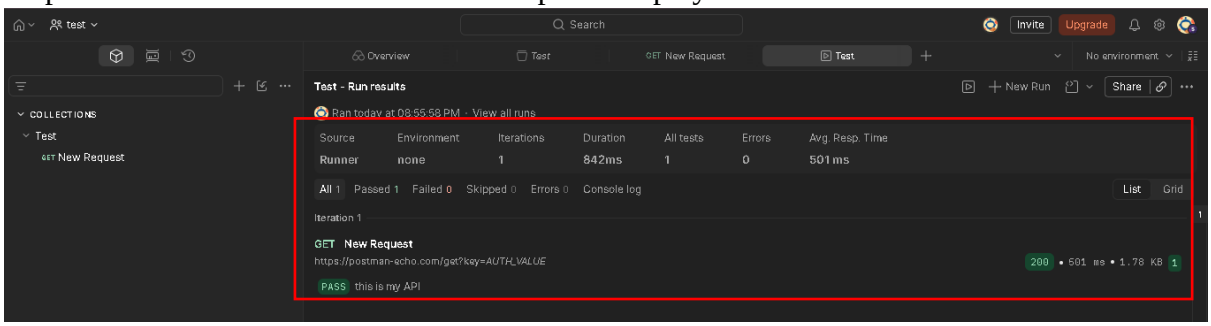
Step 9: Click the 'Test Results' tab to view the outcome. Then, right-click on the 'TEST' collection from the sidebar and drag it into the Runner window, then click 'Run on TEST'.



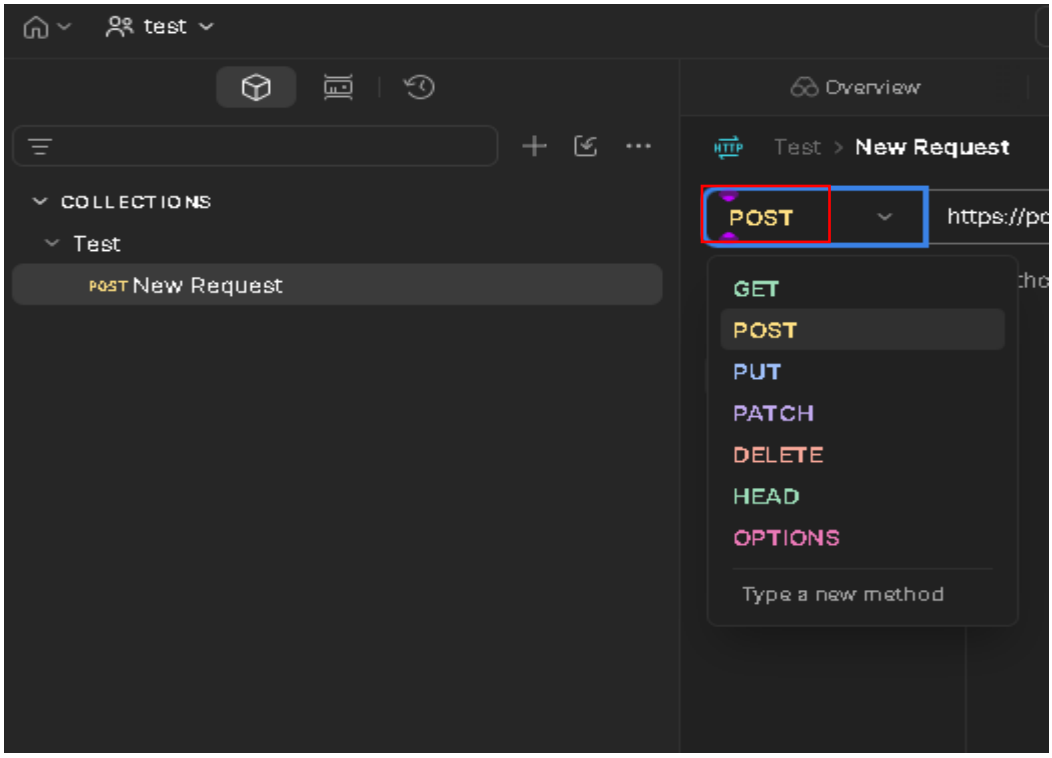


Step 10: Then click on ‘Run Test’ here.

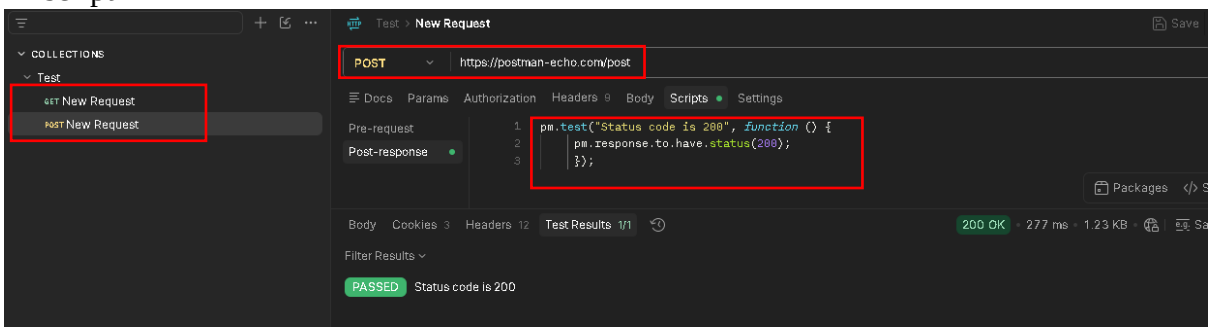
Step 11: Now the TEST – Run results output is displayed as shown below.



Step 12: Now select ‘Post’ option & select ‘POST’ method here.

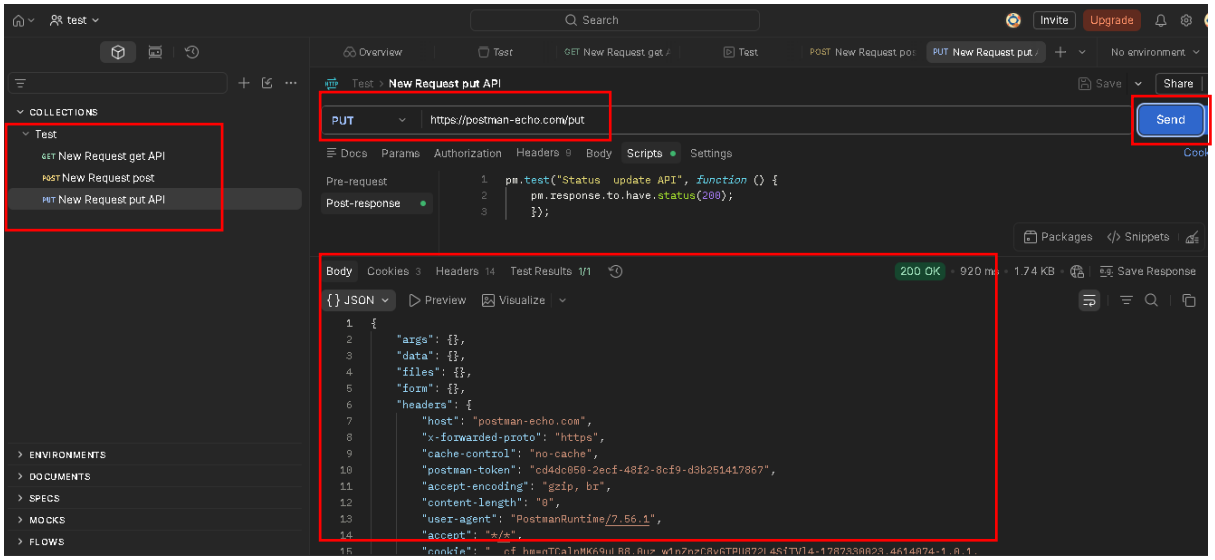


Step 13: Now select 'POST' method & click on the HTTP link as shown below & then go to → script →Enter the below code → Save it & Send.

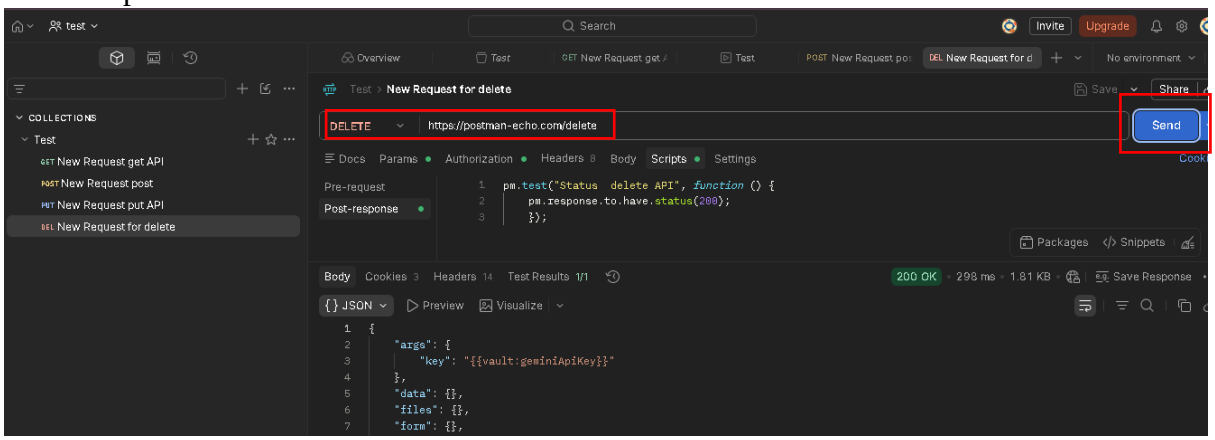


Now perform the same procedures for other two HTTP methods as shown below.

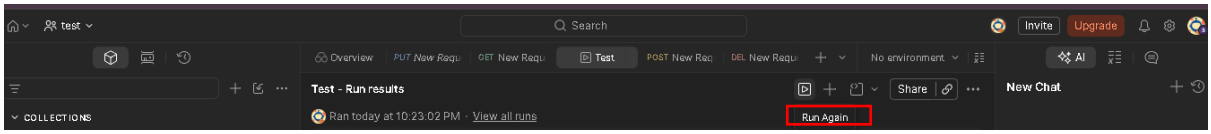
Step 14: Now select 'PUT' method & click on the HTTP link as shown below & then go to → scripts →Enter the below code → Save it & Send.

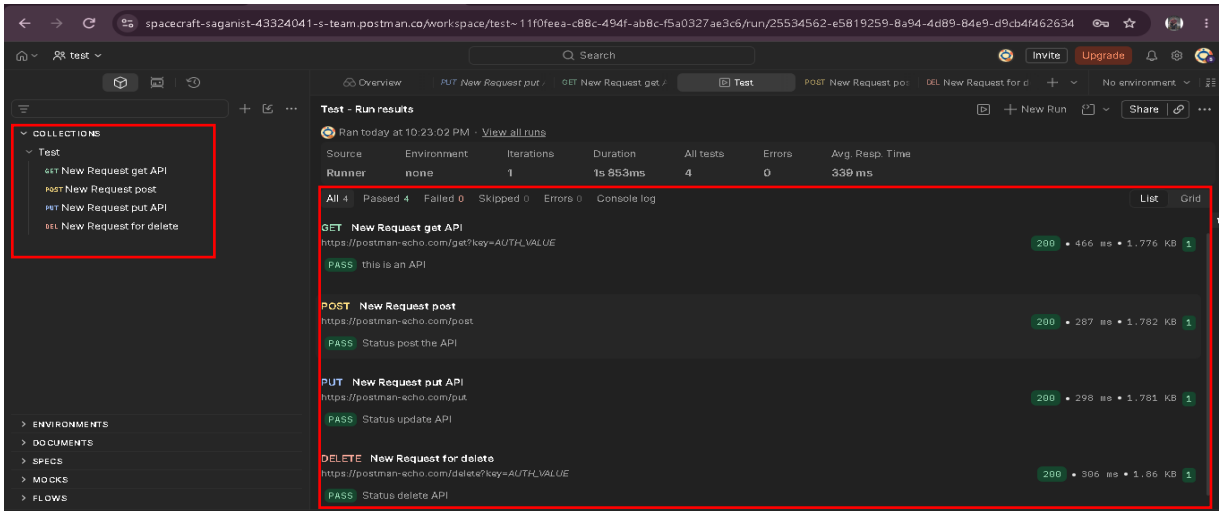


Step 15: Now select 'DELETE' method & click on the HTTP link as shown below & then go to → script →Enter the below code → Save it & Send.



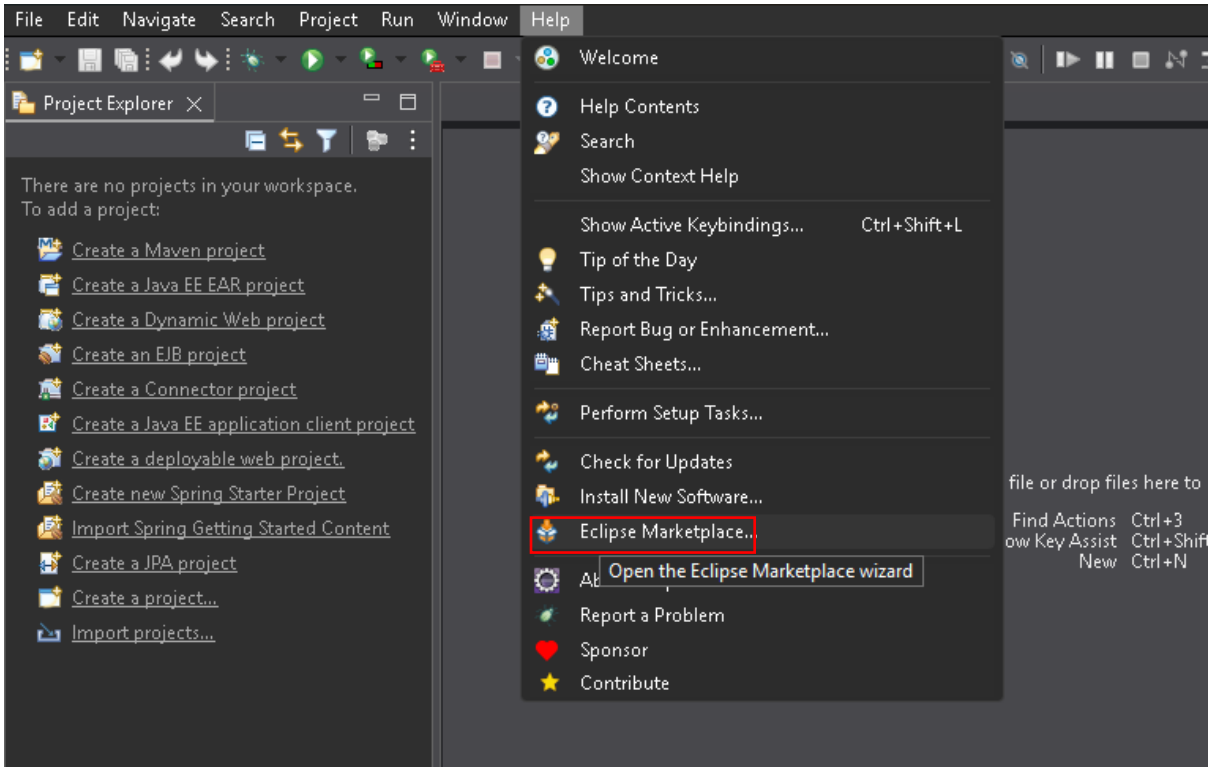
Step 16: Now the TEST – Run results outputs are displayed as shown below.



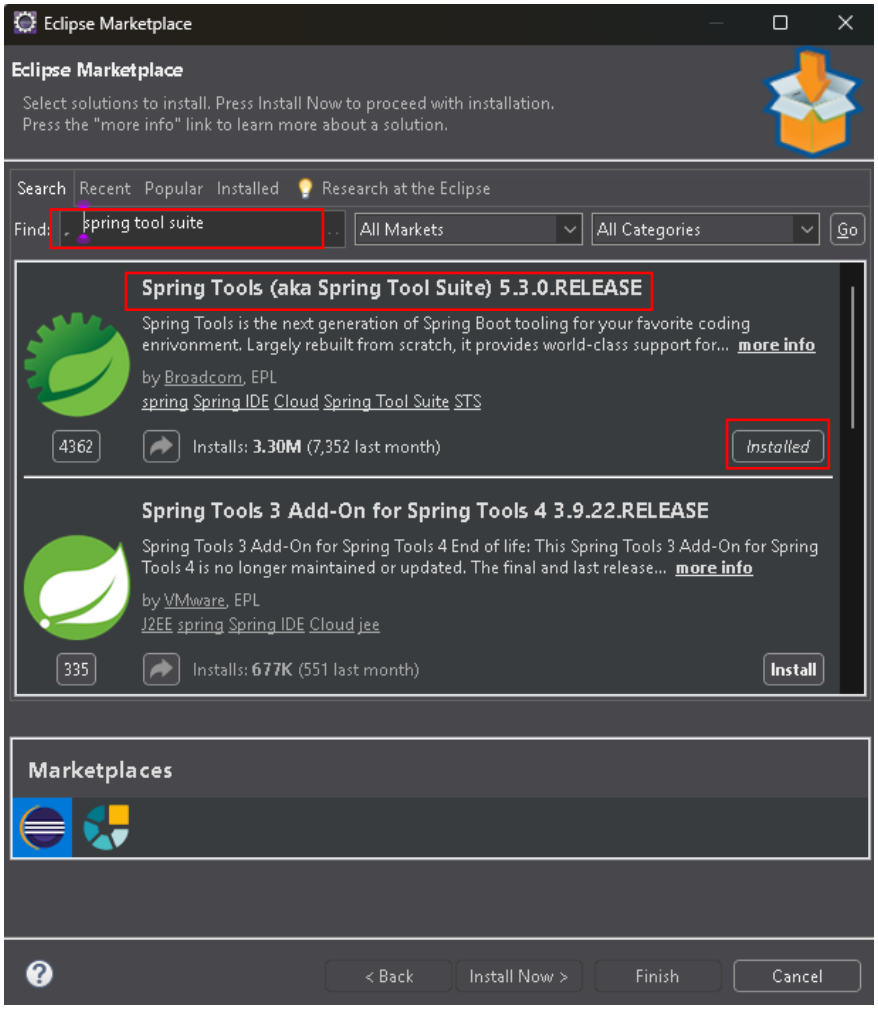


2. Create REST controller for CRUD operations Versioning Spring REST APIs

Step 1: Open ‘Eclipse IDE for Enterprise edition’ & go to -⟶ Help → Eclipse Marketplace as shown below.

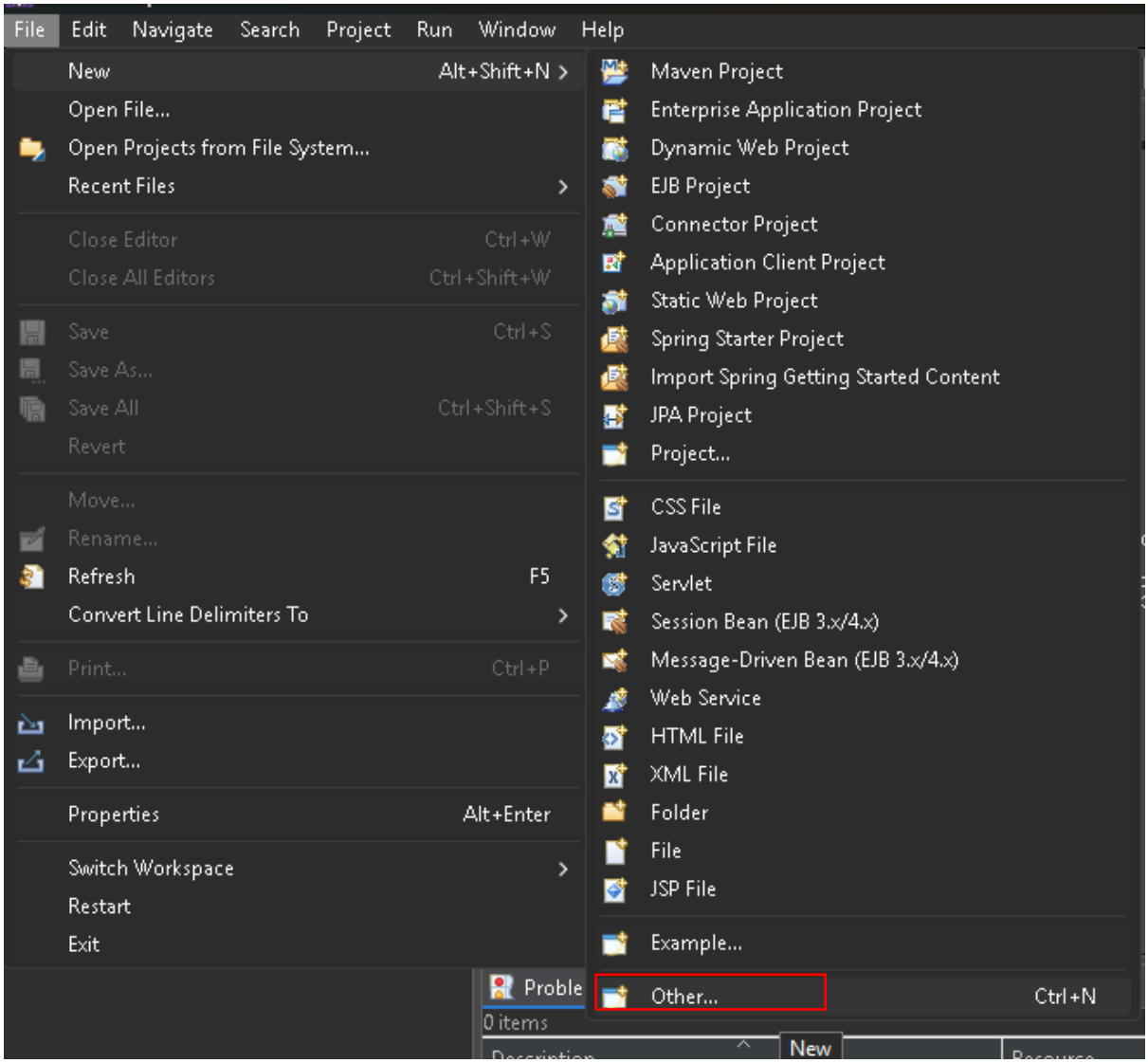


Step 2: Now search for ‘Spring tool suite’ & click on ‘Install’ to install the latest version.

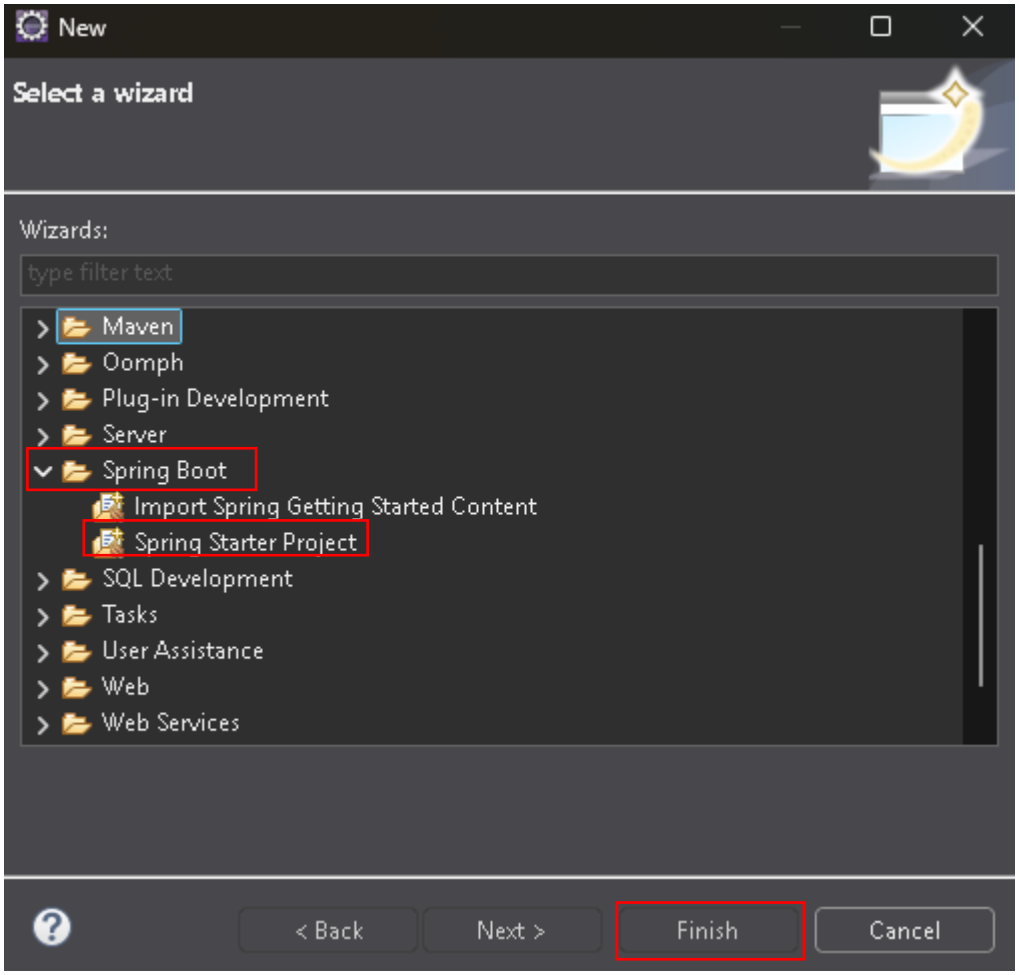


Configure using SpringBoot application property file

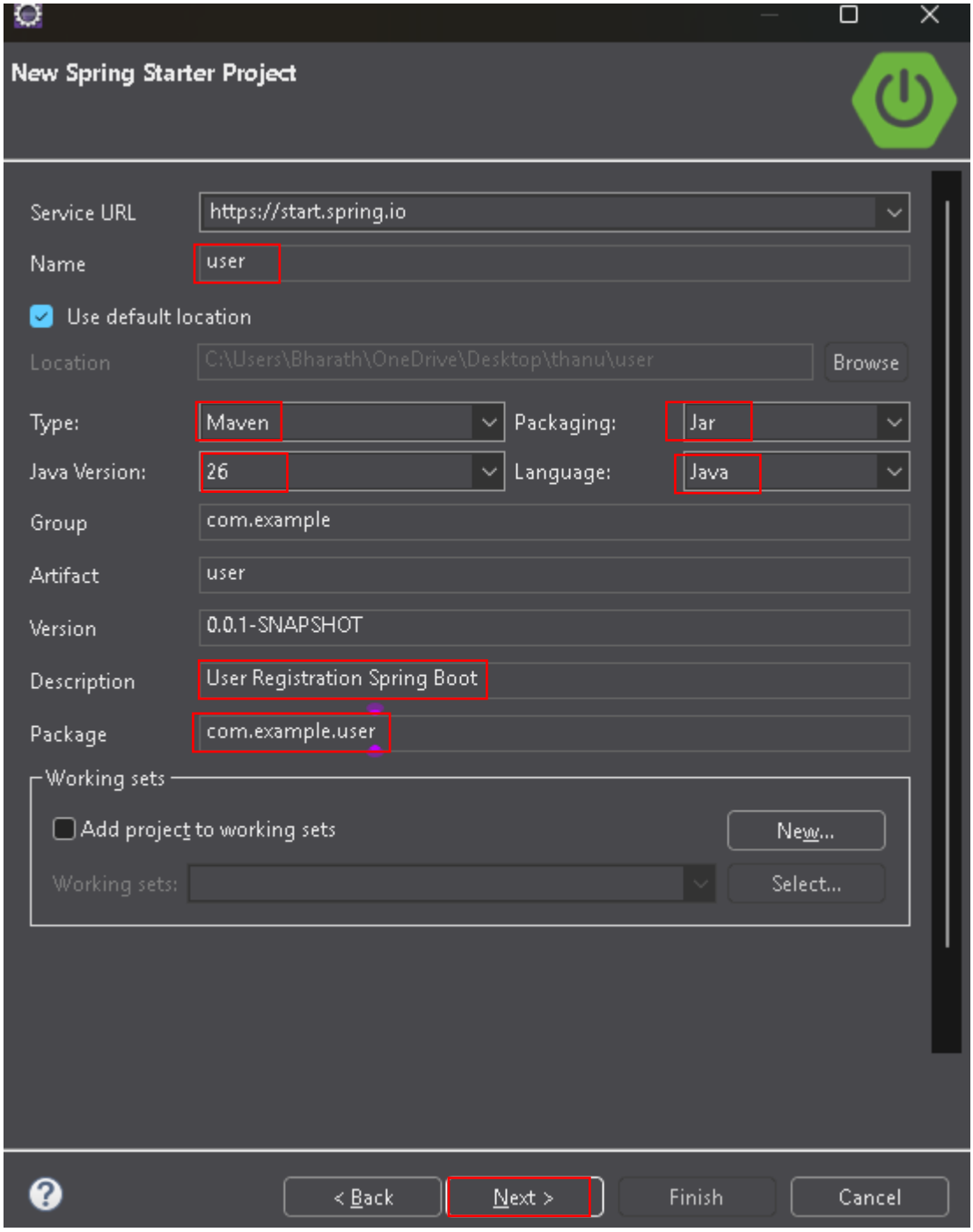
Step 3: After the installation is complete, go to → File → New → Other option as shown below.



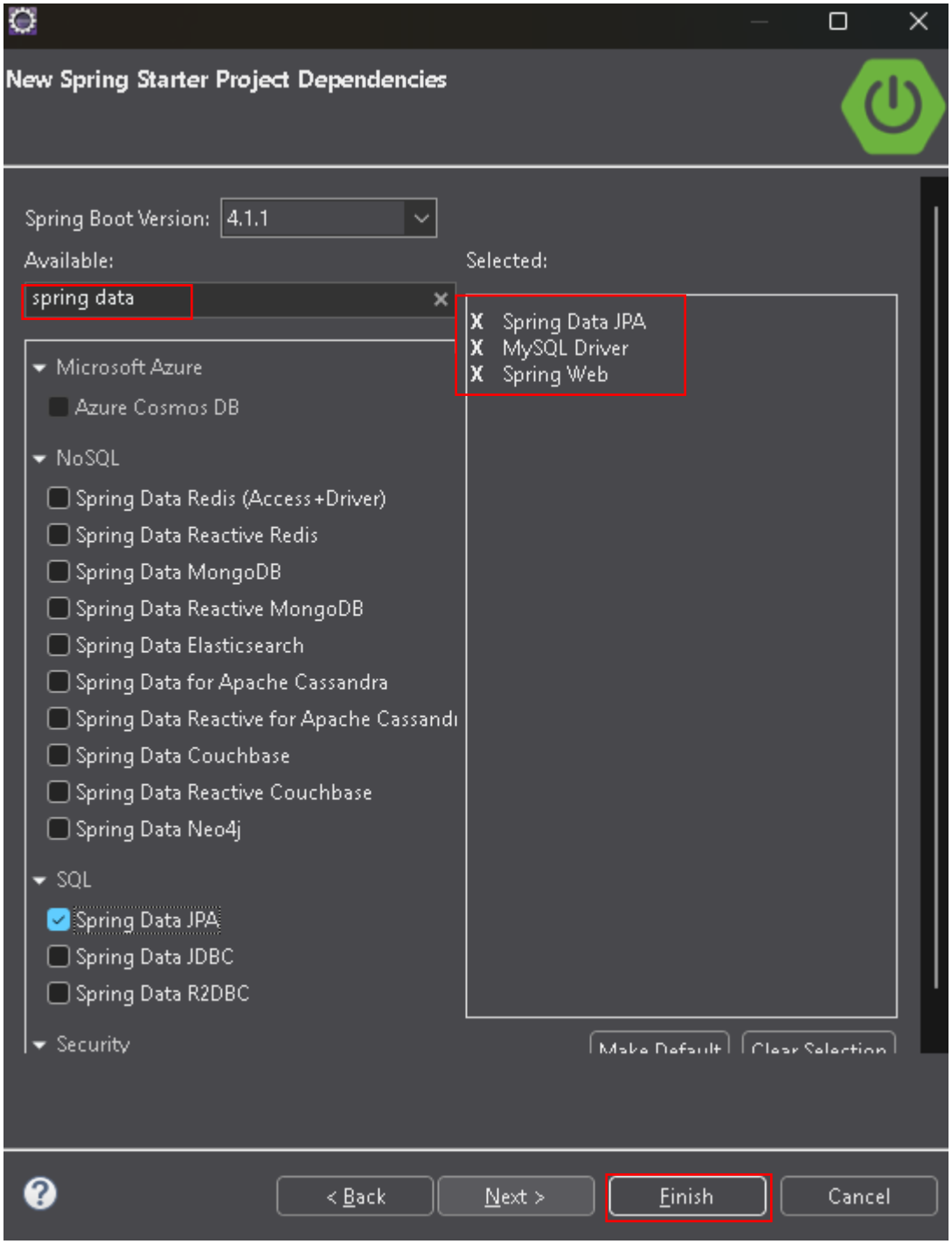
Step 4: Now select ‘Spring Starter Project’ & click on ‘Next’ option.



Step 5: Give the Name as ‘User’ → select ‘Maven’ → Description as User Registration Spring BootApplication → Package as ‘com.example.user’ & then click on ‘Next’.



Step 6: Now add all the required dependencies as shown below & click on ‘Finish’.



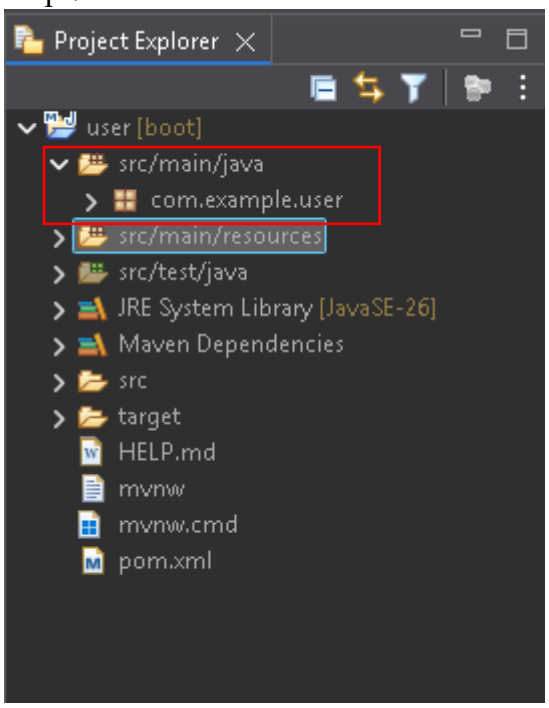
Spring REST – creating Spring REST controller

- Controller Layer (handling request and responses)

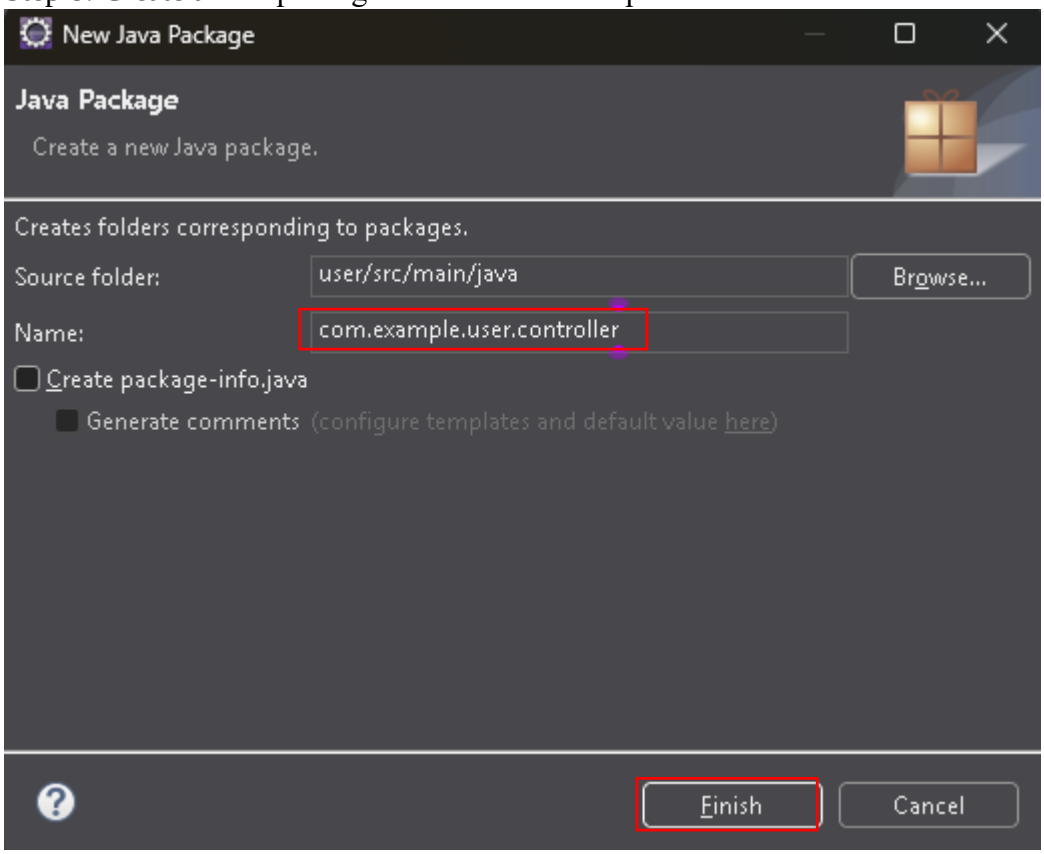
- Service Layer (Application business logic)

- Repository layer (Communicate with DB)

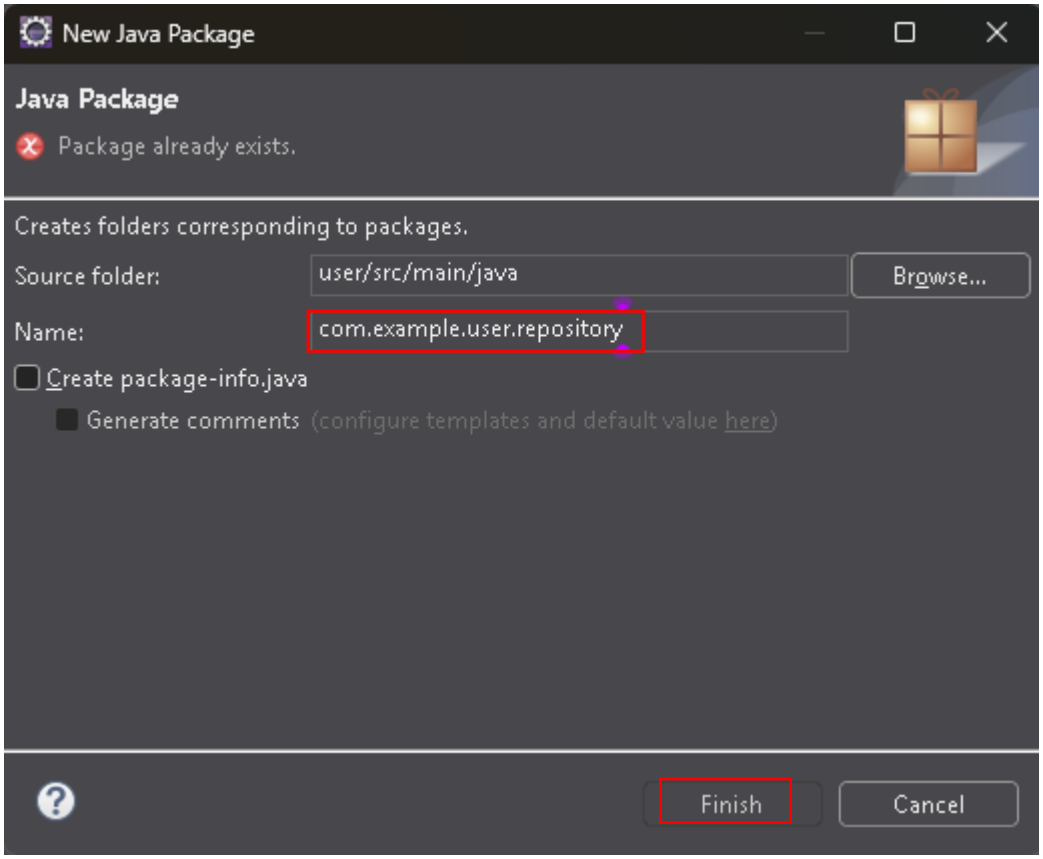
Step 7: Then select the ‘src’ folder New Package.



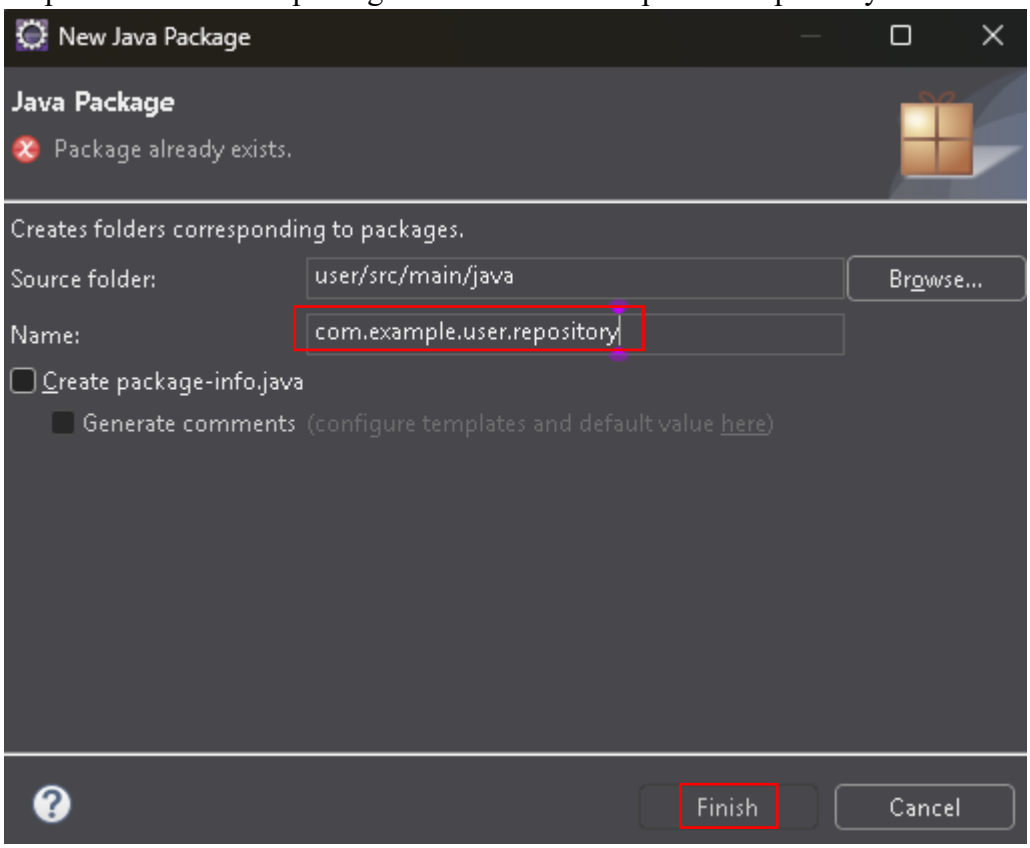
Step 8: Create a new package named ‘com.example.user.controller’ & click on ‘Finish’.



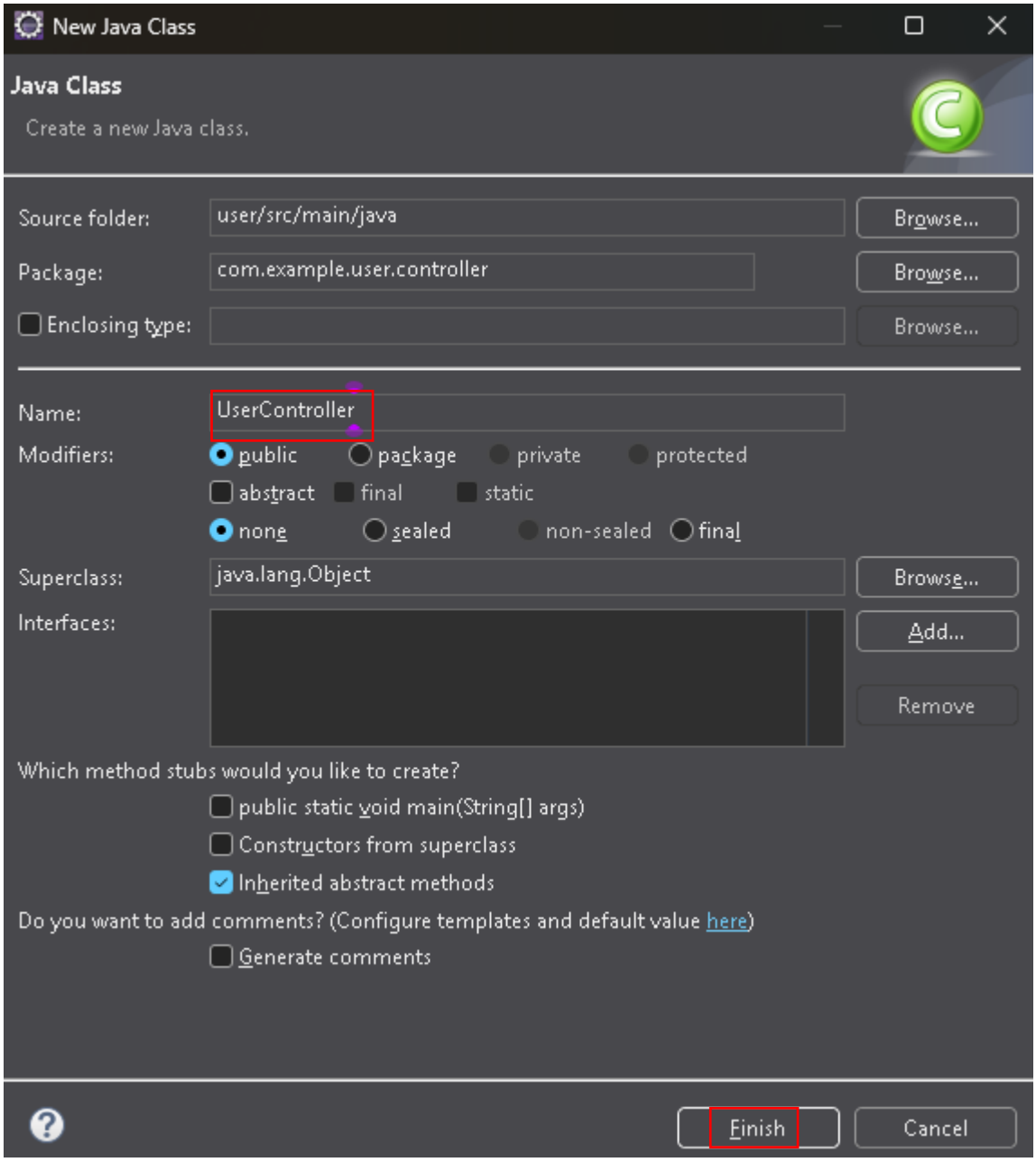
Step 9: Create a new package named ‘com.example.user.entity’ & click on ‘Finish’.



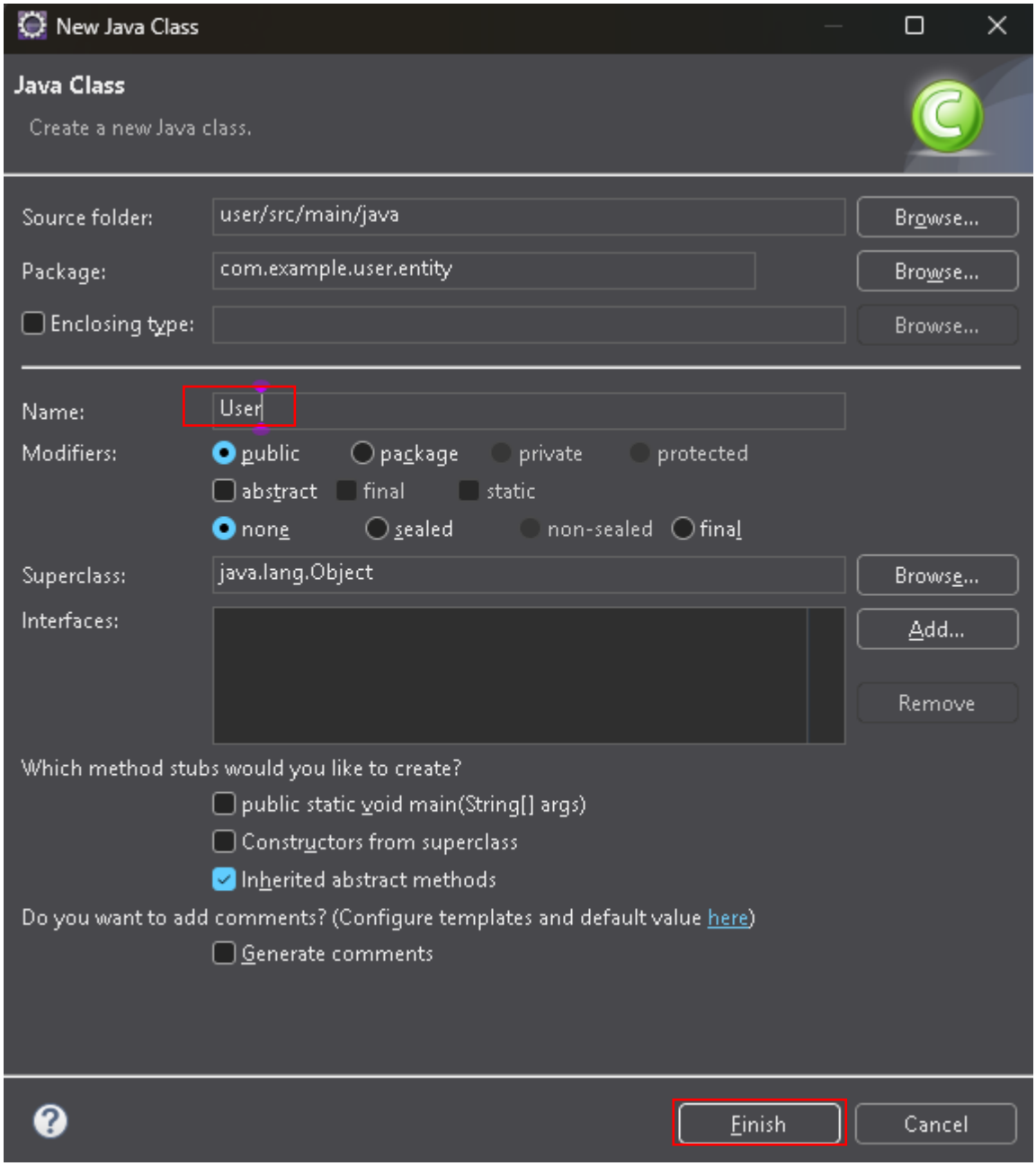
Step 10: Create a new package named 'com.example.user.repository' & click on 'Finish'.



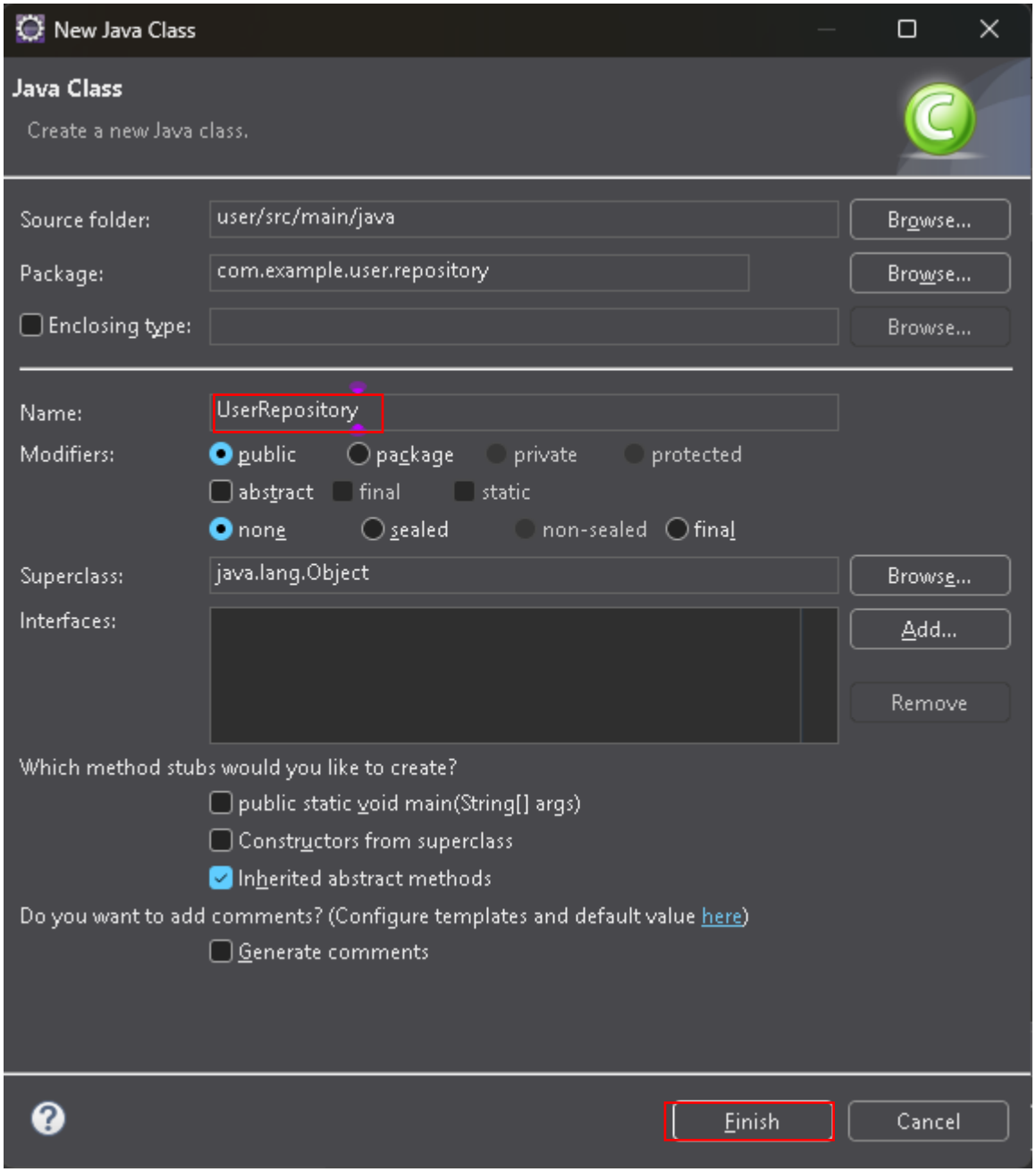
Step 11: Now right click on the 'com.example.user.controller' package New Class & create a new class named 'UserController' & then click on 'Finish'.



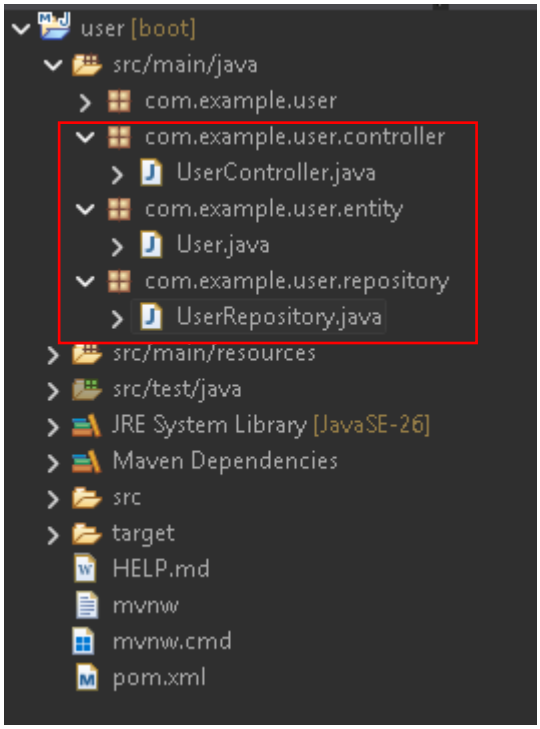
Step 12: Now right click on the ‘com.example.user.entity’ package New Class & create a new class named ‘User’ & then click on ‘Finish’.



Step 13: Now right click on the 'com.example.user.repository' package New Interface & create a new class named 'UserRepository' & then click on 'Finish'.



Step 14: All the created packages and classes are displayed here.



Step 15: Now add code to UserController.java file as shown below.

UserApplicati... | UserControll... | User.java | UserReposito...

```java
package com.example.user.controller;

import com.example.user.entity.User;
import com.example.user.repository.UserRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/users")
public class UserController {

    private final UserRepository userRepository;

    public UserController(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @GetMapping
    public List<User> getAllUser() {
        return userRepository.findAll();
    }

    @GetMapping("/{id}")
    public User getUserById(@PathVariable long id) {
        return userRepository.findById(id).orElseThrow();
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userRepository.save(user);
    }

}
```

```
33
34      @PutMapping("/{id}")
35      public User updateUser(@RequestBody User user, @PathVariable long id) {
36          User ex = userRepository.findById(id).orElseThrow();
37          ex.setFirstname(user.getFirstname());
38          ex.setLastname(user.getLastname());
39          return userRepository.save(ex);
40      }
41
42      @DeleteMapping("/{id}")
43      public ResponseEntity<Void> deleteUser(@PathVariable long id) {
44          userRepository.deleteById(id);
45          return ResponseEntity.ok().build();
46      }
47  }
```

Step 16: Now add code to User.java file as shown below.

UserApplicati... | UserControll... | User.java | UserReposito... | application....

```
1  package com.example.user.entity;
2
3  import jakarta.persistence.*;
4
5  @Entity
6  @Table(name = "users")
7  public class User {
8
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private long id;
12
13     private String firstname;
14     private String lastname;
15
16     public long getId() { return id; }
17     public void setId(long id) { this.id = id; }
18     public String getFirstname() { return firstname; }
19     public void setFirstname(String firstname) { this.firstname = firstname; }
20     public String getLastname() { return lastname; }
21     public void setLastname(String lastname) { this.lastname = lastname; }
22 }
```

Step 17: Now add code to UserRepository.java file as shown below.

UserApplicati... | UserControll... | User.java | UserReposito... | app

```
1  package com.example.user.repository;
2
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import com.example.user.entity.User;
5
6  public interface UserRepository extends JpaRepository<User, Long> {}
7
```

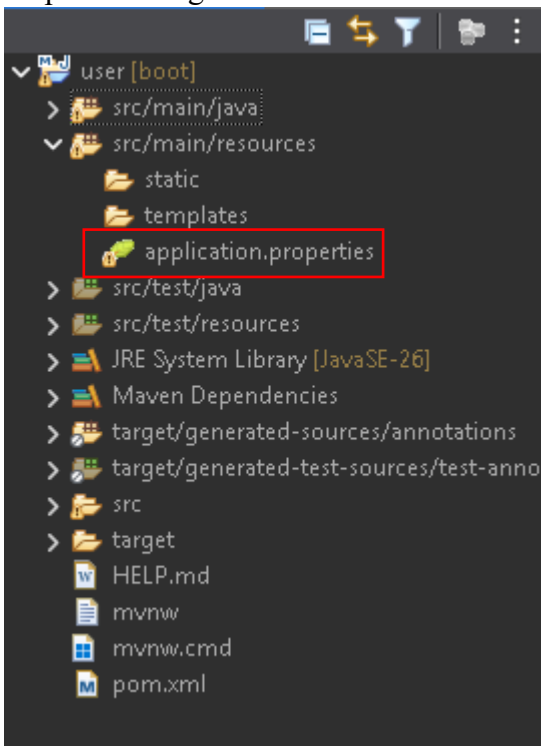
```java
package com.example.user.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.example.user.entity.User;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
}
```

Step 18: Now go to 'src/main/resources' folder select 'application.properties' file here.

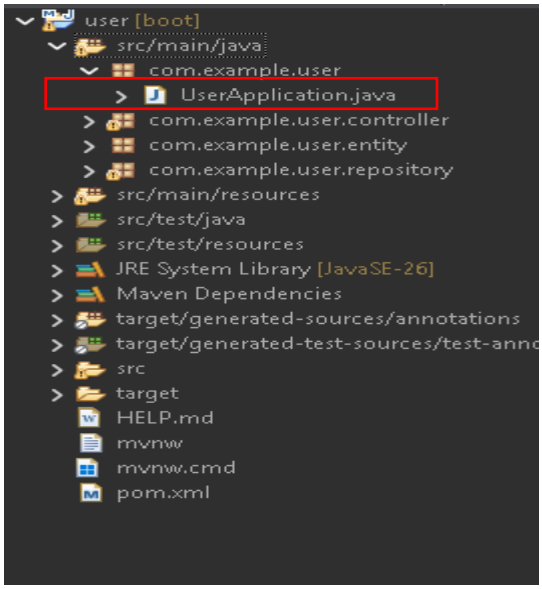


Step 19: Now add code to application.properties file as shown below.

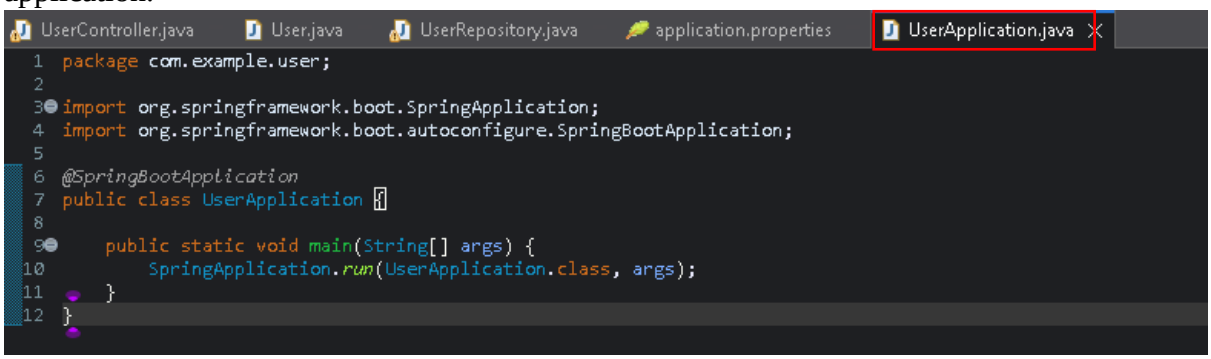
```properties
spring.application.name=user-1

# MySQL Connection Settings
spring.datasource.url=jdbc:mysql://localhost:3306/data3
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
```

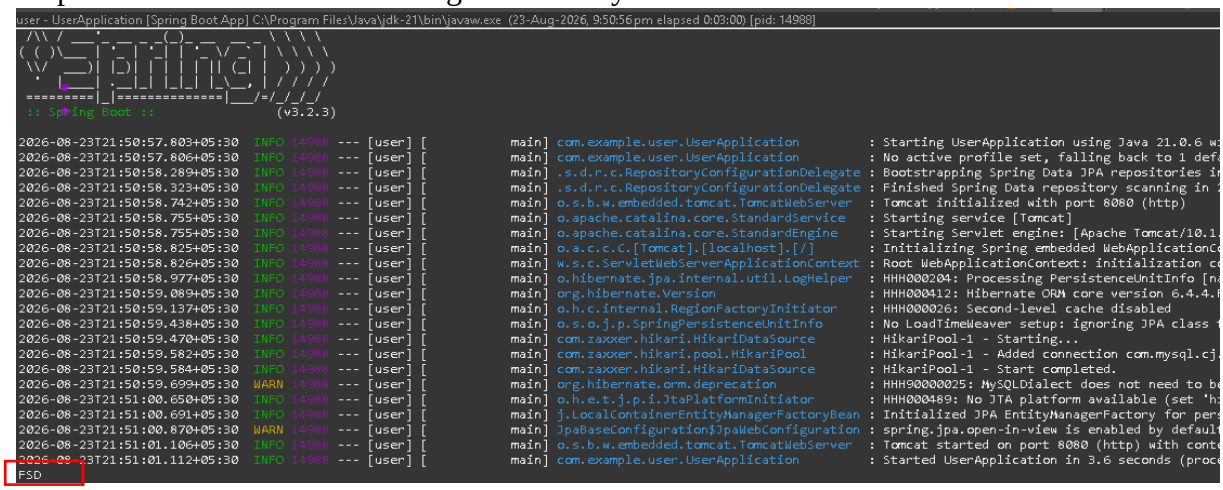
Step 20: Now add code to UserApplication.java file which includes the main method as shown below.



Step 21: Go to src/main/java right click on the UserApplication.java Run as 1SPRING BOOT application.



Step 22: Now the server is running successfully.



• *Creation of database using MySQL Command Line Client*

Step 23: Enter the password created→ give the command as show databases; to view all the inbuilt databases →now enter the command as create database data1; to create a new database with any name→ again enter show databases; to view the newly created database as shown below.

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 39
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

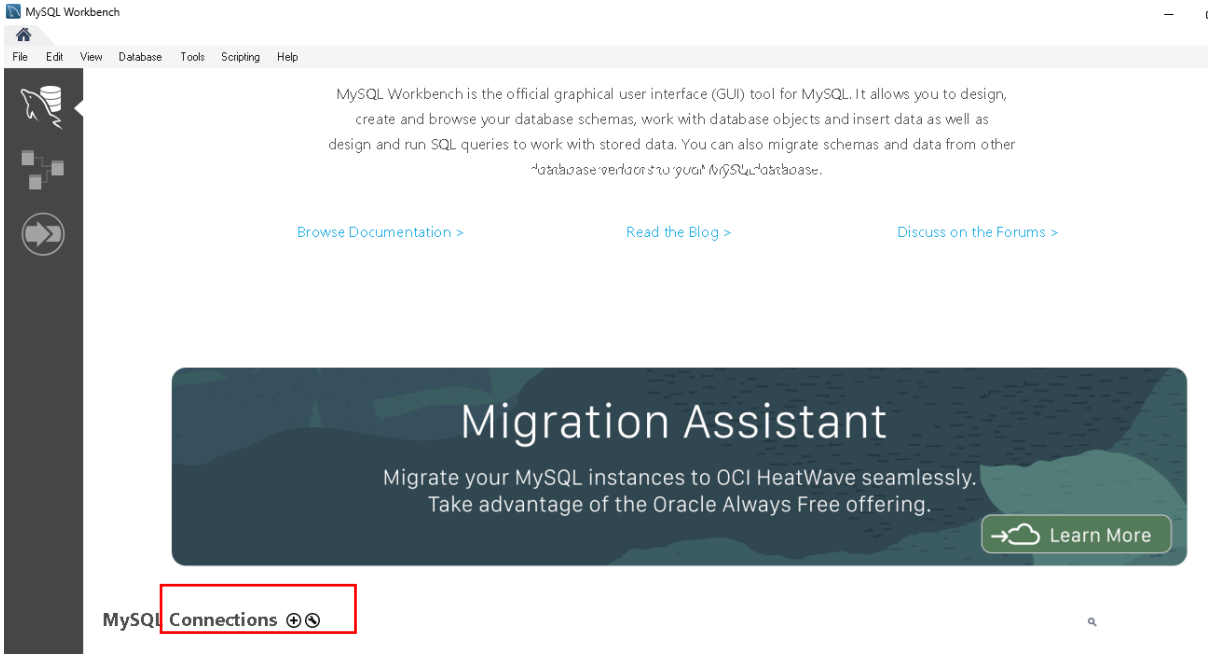
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database data1;
Query OK, 1 row affected (0.03 sec)

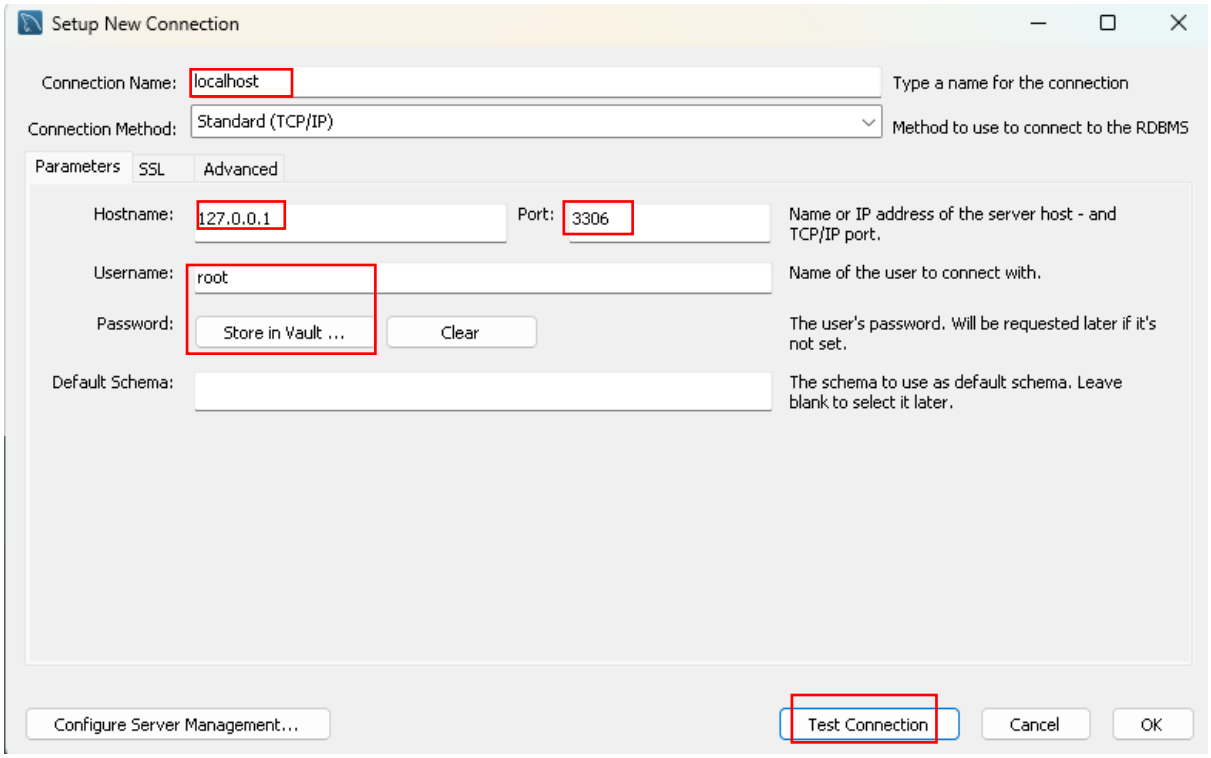
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| data1              |
| employee           |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| user_db            |
+--------------------+
7 rows in set (0.01 sec)
```

Connection of newly created database to the MySQL Workbench.

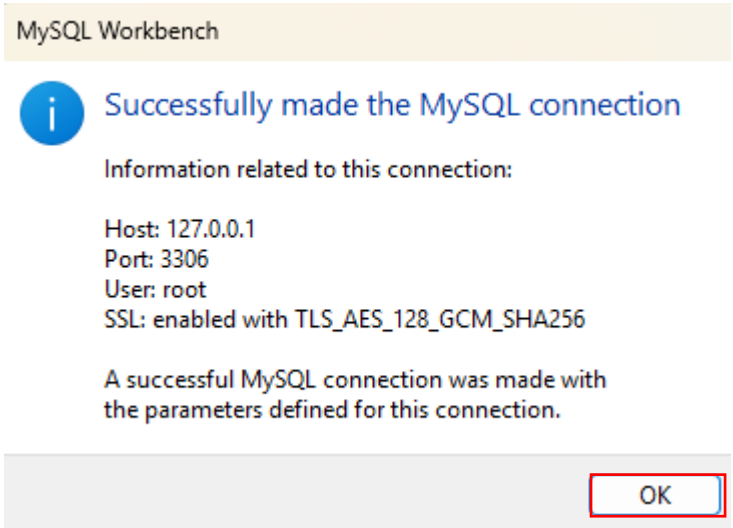
Step 24: Go to '+' option to add new connection.



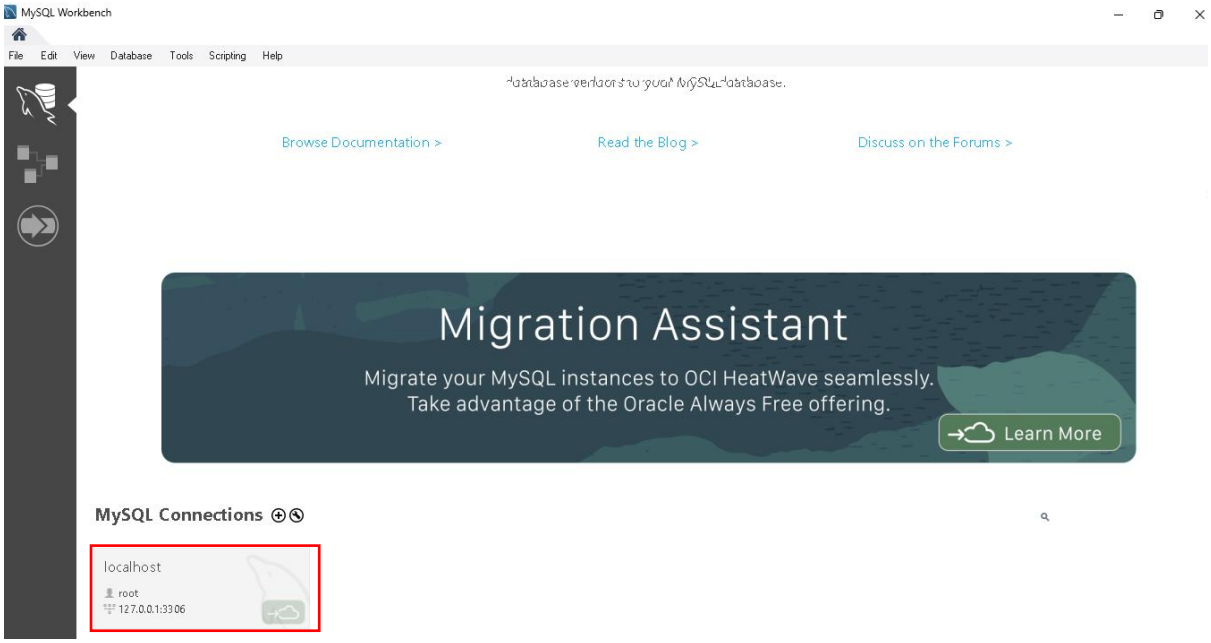
Step 25: Give a name to your connection as 'MyConnection' select the Hostname as 'localhost' or select the default ip address of your local computer give the port no. which you've entered while installation of MySQL Workbench (3306) Username as 'root'.



Step 26: Then enter the Password – which was created while installing the MySQL Workbench & click on‘OK’ & then click on ‘Test connection’.

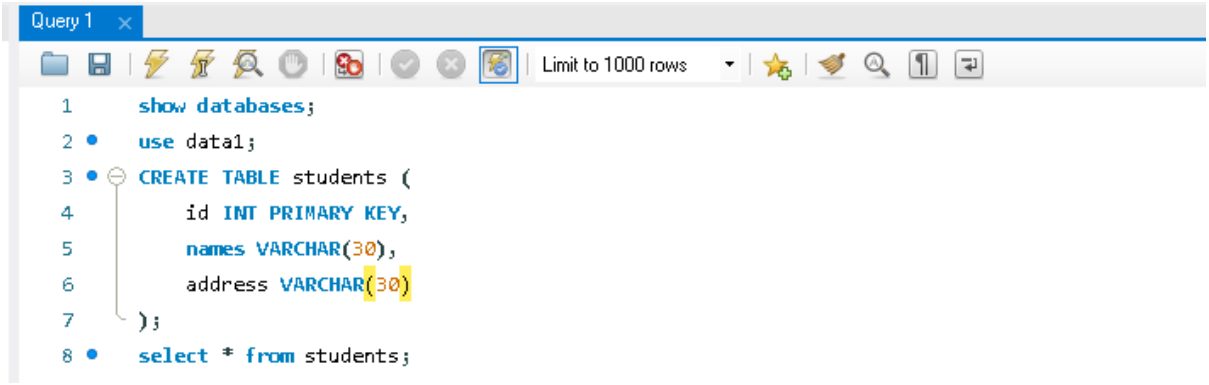


Step 27: Now click on ‘OK’ option as shown below.

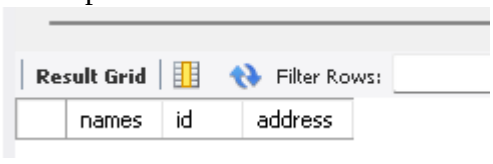


• Creation of simple table.

Step 28: Now enter the below code to create a simple table as shown below.

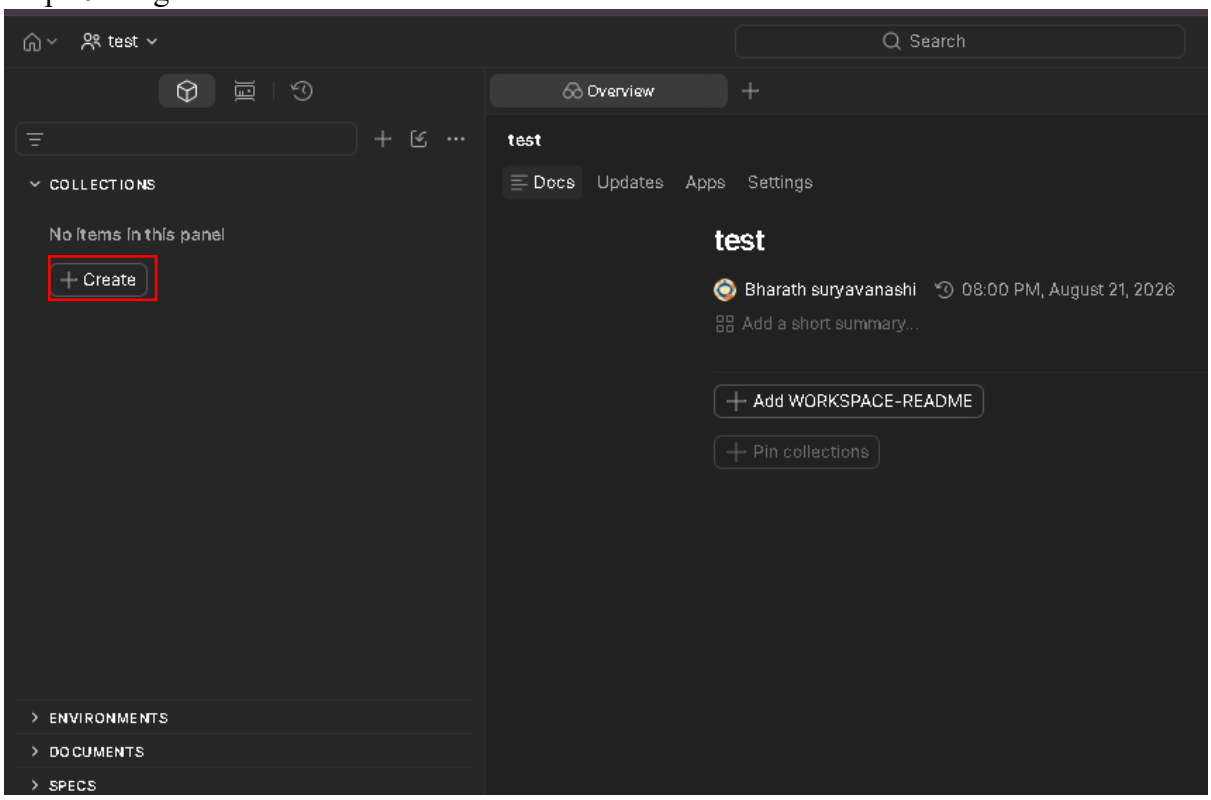


Note: To run the code, select each code line separately & press ctrl+shift+enter key to view the output.



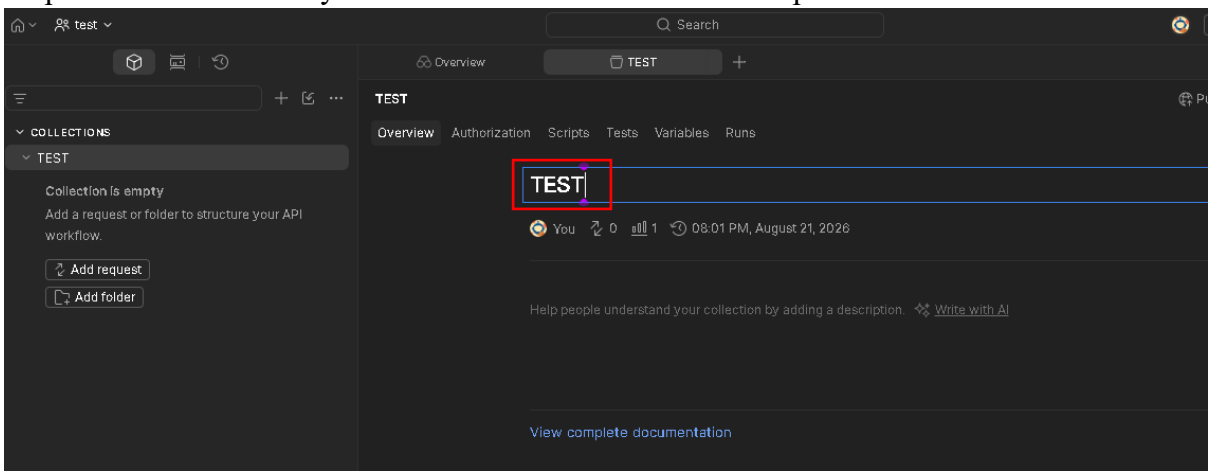
• *Performing CRUD operations using Postman RESTful Service.*

Step 29: Login to the Postman & click on ‘Create collection’ here.



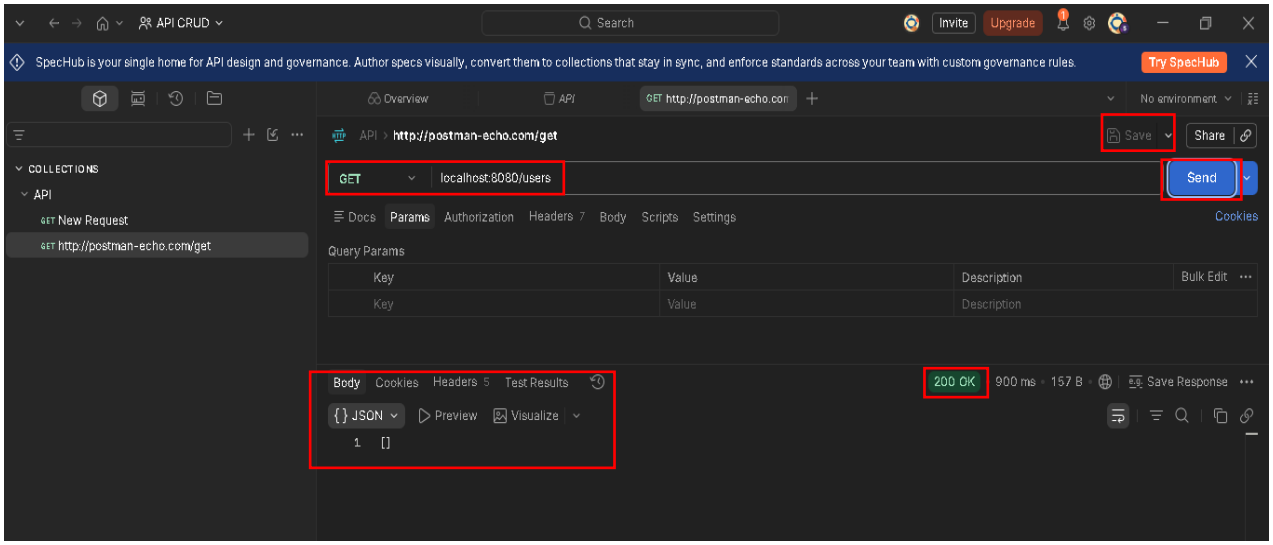
Step 30: Again click on ‘Create a collection’ here.

Step 31: Give a name to your collection & click on ‘Create’ option here.



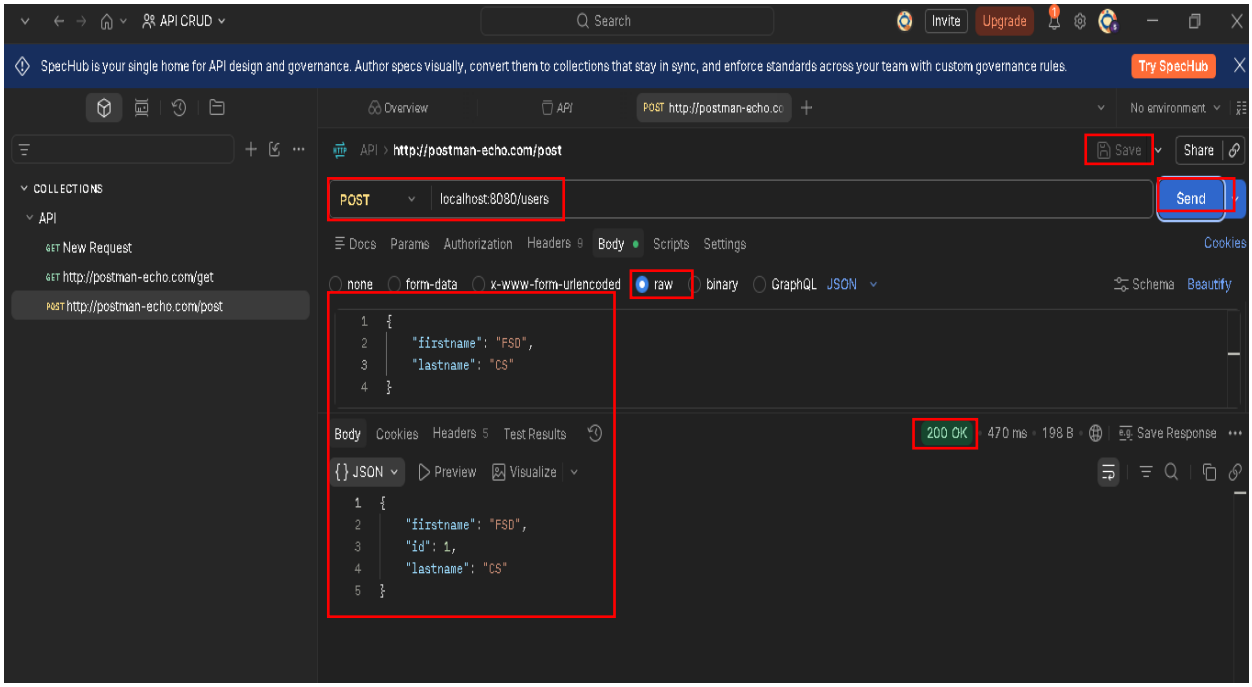
Step 32: Then click on ‘Save’.

Step 33: Select ‘GET’ method & enter the command as ‘localhost:8080/users’ Save it & Send.



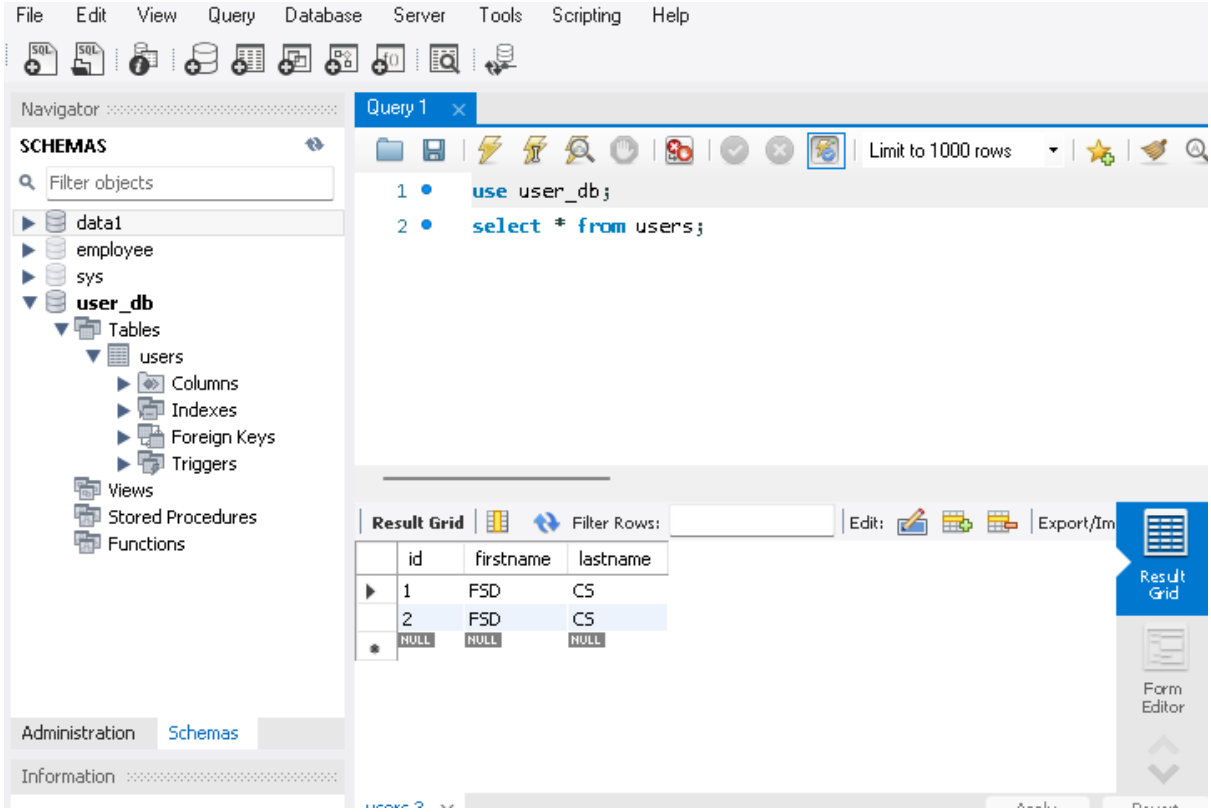
Step 34: Select ‘POST’ method.

Step 35: Then enter the command as ‘localhost:8080/users’ select Body Text type - JSON enter the below code as shown below Save it & Send.



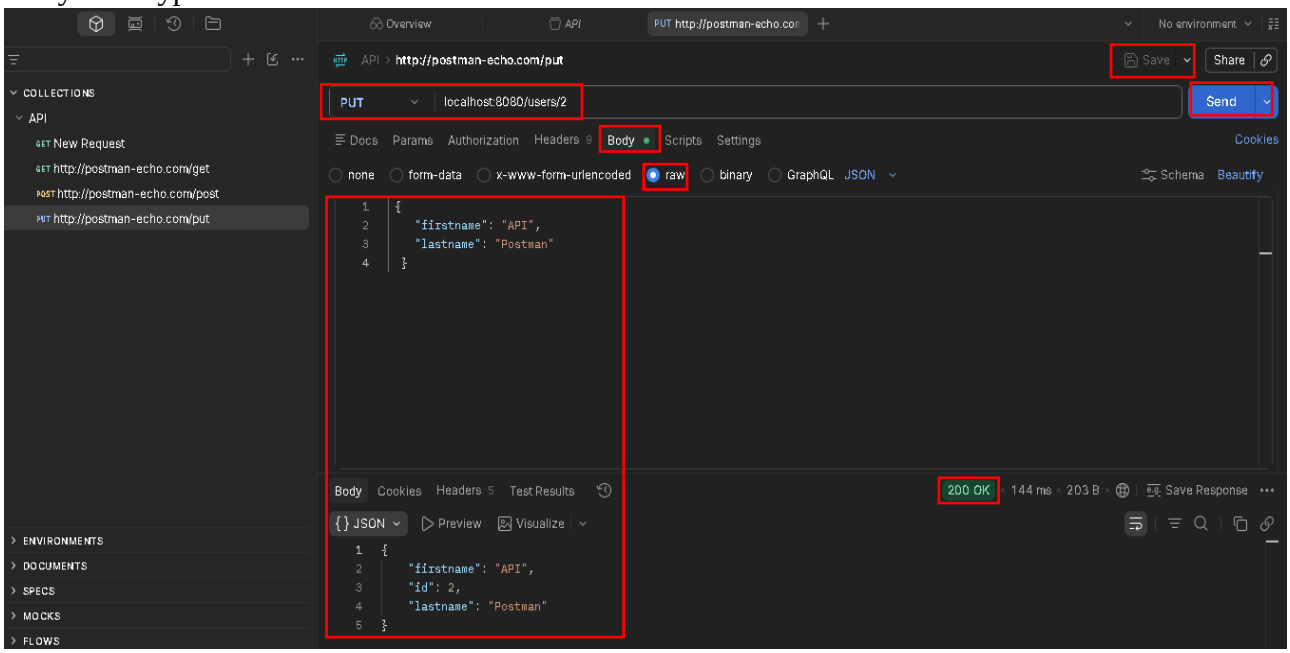
Step 36: Now go to MySQL Workbench & enter the below command as shown below. To

run the code,select each code line separately & press ctrl+shift+enter key to view the output.



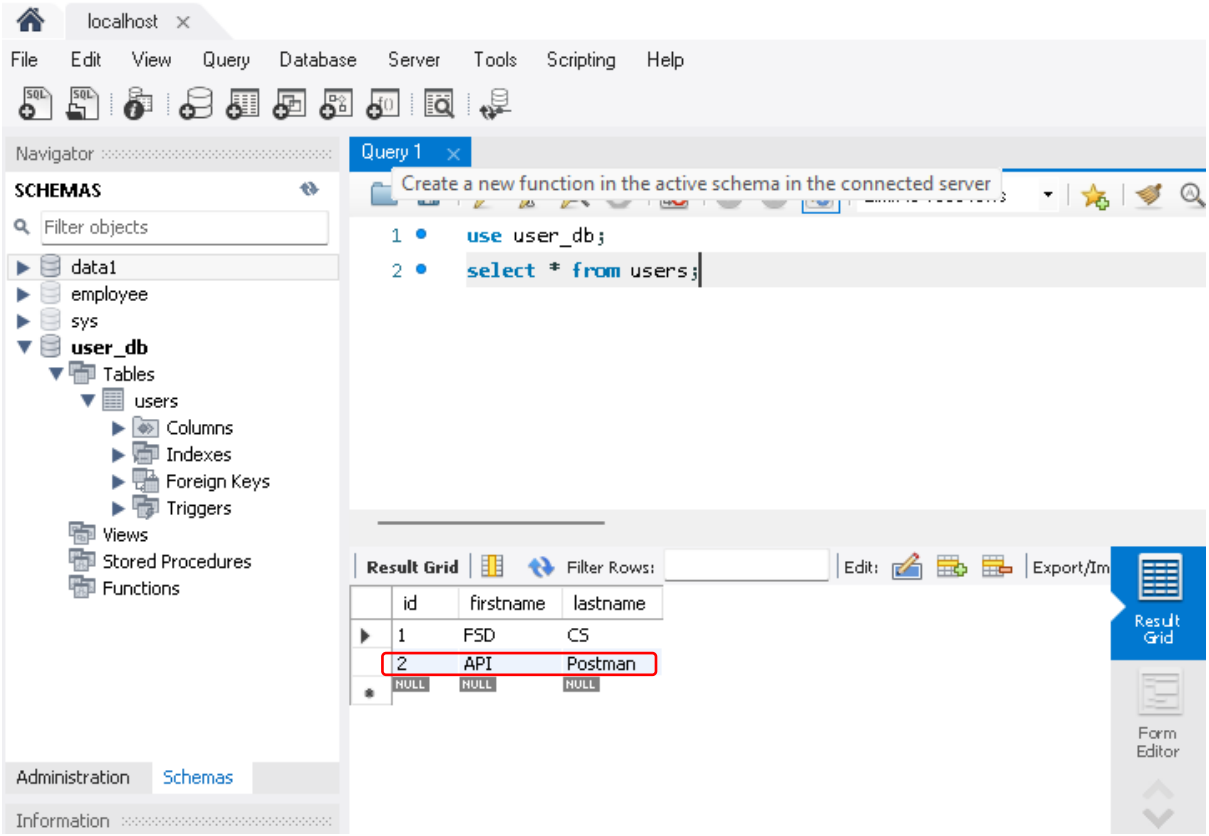
Note: Here the POST method is created successfully.

Step 37: Select ‘PUT’ method & then enter the command as ‘localhost:8080/users/2’ select Body Text type - JSON enter the below code as shown below Save it & Send.



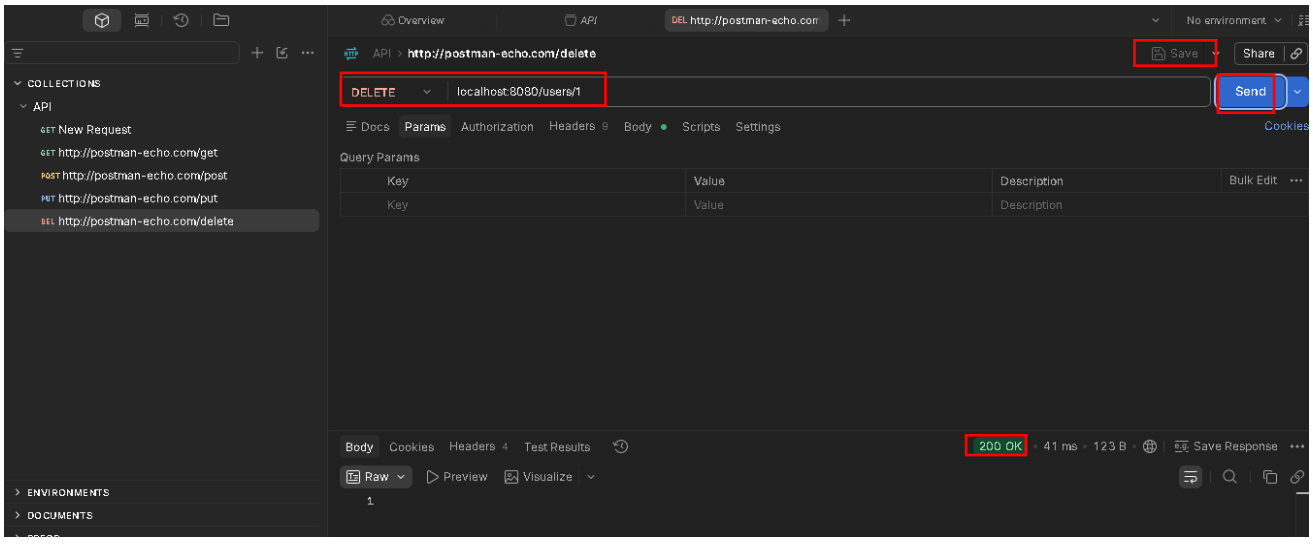
Step 38: Now go to MySQL Workbench & enter the below command as shown below. To

run the code,select each code line separately & press ctrl+shift+enter key to view the output.



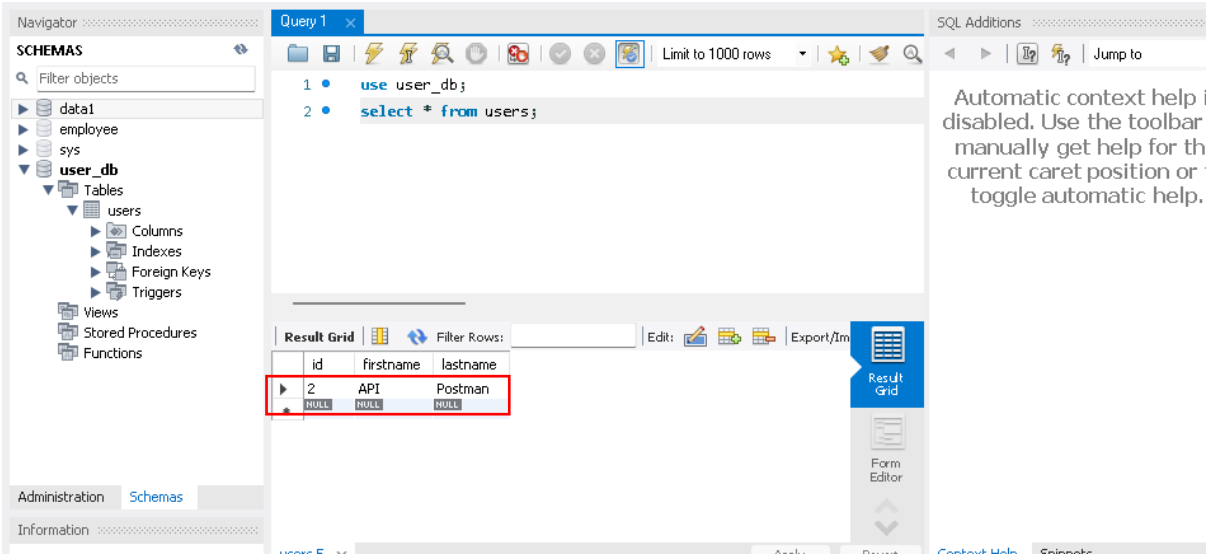
Note: Here the PUT method is updated successfully.

Step 39: Select 'DELETE' method & then enter the command with the id no. specified as'localhost:8080/users/1' → select Body → Text type - JSON → enter the below code as shown below →Save it & Send.



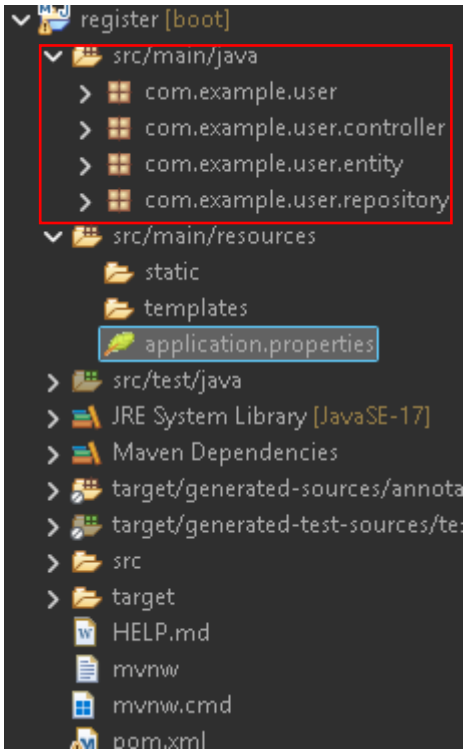
Step 40: Now go to MySQL Workbench & enter the below command as shown below. To run the code,select each code line separately & press ctrl+shift+enter key to view the output.

Note: Here the DELETE method is deleted successfully.



3. Create user registration form.

Step 1: Now go to 'Eclipse IDE for enterprise edition' → File → New → Spring starter project → Add all the dependencies → Create three packages named as (com.example.user.controller,com.example.user.entity, com.example.user.repository) & create two classes named asUserController.java, User.java files & then create interface named as UserRepository.java file as shown below.



UserApplication.java | UserController.java | User.java

```java
package com.example.user;

import org.springframework.boot.SpringApplication;

@SpringBootApplication
public class UserApplication {

    public static void main(String[] args) {
        SpringApplication.run(UserApplication.class, args);
        System.out.println("welcome to FSD");   }

}
```

Step 2: Now add code to UserController.java file as shown below.

UserApplication.java | UserController.java | User.java

```java
package com.example.user.controller;

import com.example.user.entity.User;
import com.example.user.repository.UserRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/users")
public class UserController {

    private final UserRepository userRepository;

    public UserController(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @GetMapping
    public List<User> getAllUser() {
        return userRepository.findAll();
    }

    @GetMapping("/{id}")
    public User getUserById(@PathVariable long id) {
        return userRepository.findById(id).orElseThrow();
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userRepository.save(user);
    }
```

```
    }

    @PutMapping("/{id}")
    public User updateUser(@RequestBody User user, @PathVariable long id) {
        User existingUser = userRepository.findById(id).orElseThrow();
        existingUser.setFirstname(user.getFirstname());
        existingUser.setLastname(user.getLastname());
        existingUser.setRegisternumber(user.getRegisternumber());
        existingUser.setDOB(user.getDOB());
        existingUser.setAddress(user.getAddress());
        existingUser.setDepartment(user.getDepartment());
        return userRepository.save(existingUser);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable long id) {
        User existingUser = userRepository.findById(id).orElseThrow();
        userRepository.delete(existingUser);
        return ResponseEntity.ok().build();
    }
}
```

Step 3: Now add code to User.java file as shown below.

UserApplication.java | UserController.java | User.java | UserRepository.java | application.properties

```
package com.example.user.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "user")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String firstname;
    private String lastname;
    private String registernumber;
    private String DOB;
    private String address;
    private String department;

    public User() {}

    public User(Long id, String firstname, String lastname, String registernumber, String DOB, String address, String department) {
        this.id = id;
        this.firstname = firstname;
        this.lastname = lastname;
        this.registernumber = registernumber;
        this.DOB = DOB;
        this.address = address;
        this.department = department;
    }

```

Step 4: Now add code to UserRepository.java file as shown below.

```
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getFirstname() { return firstname; }
    public void setFirstname(String firstname) { this.firstname = firstname; }

    public String getLastname() { return lastname; }
    public void setLastname(String lastname) { this.lastname = lastname; }

    public String getRegisternumber() { return registernumber; }
    public void setRegisternumber(String registernumber) { this.registernumber = registernumber; }

    public String getDOB() { return DOB; }
    public void setDOB(String DOB) { this.DOB = DOB; }

    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
}
```

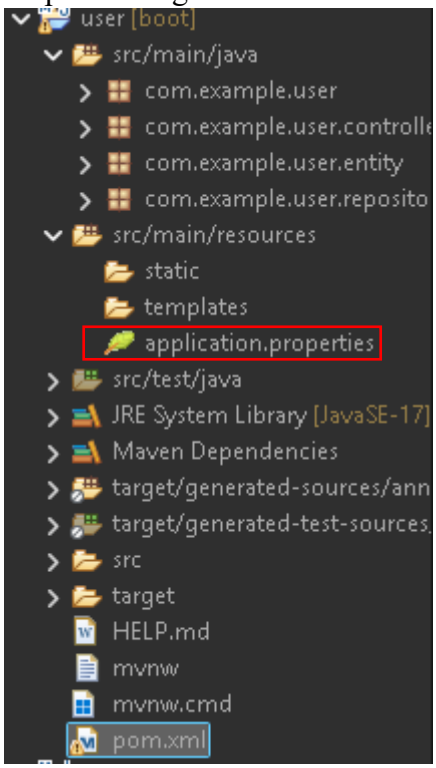
UserApplication.java | UserController.java | User.java | UserRepository.java

```
package com.example.user.repository;

import com.example.user.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
}
```

Step 5: Now go to 'src/main/resources' folder select 'application.properties' file here.



Step 6: Now add code to application.properties file as shown below.

```
spring.application.name=user
# MySQL Connection Settings
spring.datasource.url=jdbc:mysql://localhost:3306/register
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
```

Step 7: Go to src/main/java right click on the UserApplication.java Run as spring boot application.

```
package com.example.user;

import org.springframework.boot.SpringApplication;

@SpringBootApplication
public class UserApplication {

    public static void main(String[] args) {
        SpringApplication.run(UserApplication.class, args);
        System.out.println("welcome to FSD");   }

}
```

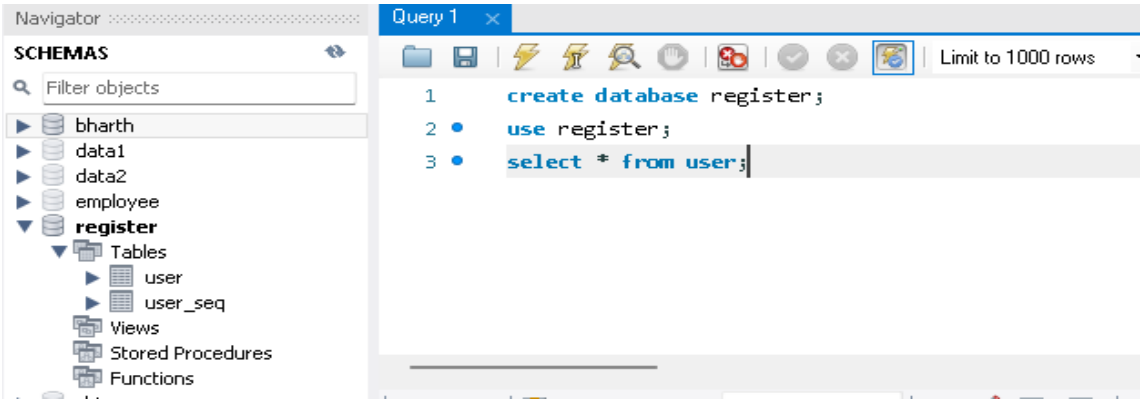
Now the server is running successfully.

```
user - UserApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe  (28-Aug-2026, 8:49:41 pm elapsed 0:27:39) [pid: 16360]

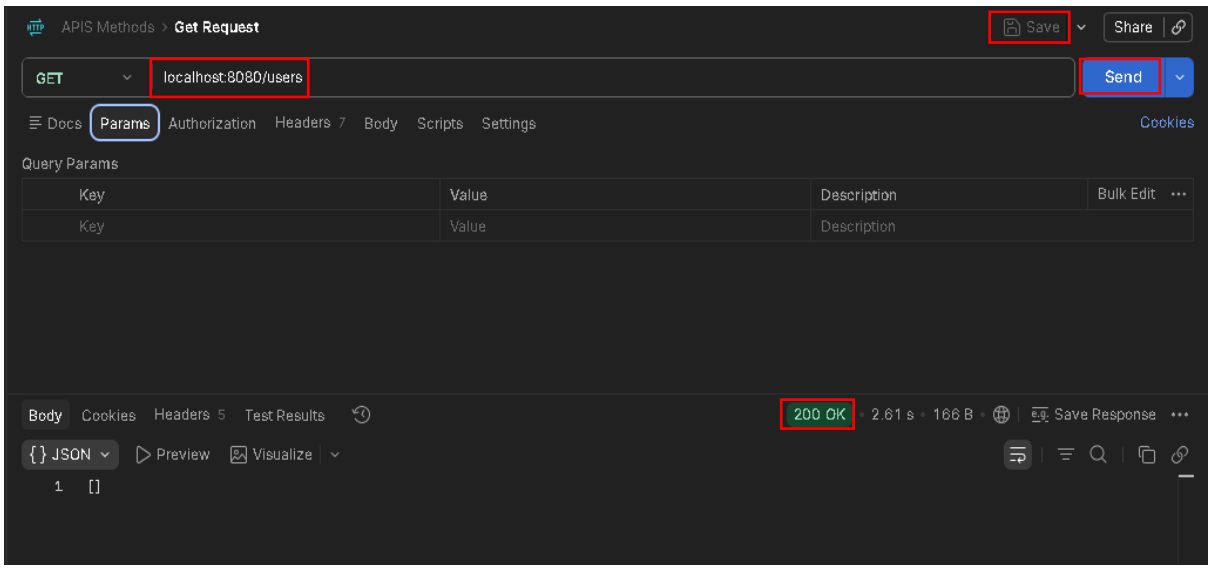
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.2.5)

2026-08-28T20:49:42.991+05:30  INFO 16360 --- [user] [           main] com.example.user.UserApplication         : Starting UserApplication us
2026-08-28T20:49:42.995+05:30  INFO 16360 --- [user] [           main] com.example.user.UserApplication         : No active profile set, fall
2026-08-28T20:49:43.825+05:30  INFO 16360 --- [user] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data J
2026-08-28T20:49:43.958+05:30  INFO 16360 --- [user] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data reposi
2026-08-28T20:49:44.980+05:30  INFO 16360 --- [user] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with por
2026-08-28T20:49:45.018+05:30  INFO 16360 --- [user] [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-08-28T20:49:45.018+05:30  INFO 16360 --- [user] [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [A
2026-08-28T20:49:45.097+05:30  INFO 16360 --- [user] [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedde
2026-08-28T20:49:45.098+05:30  INFO 16360 --- [user] [           main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext:
2026-08-28T20:49:45.251+05:30  INFO 16360 --- [user] [           main] o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing Persi
2026-08-28T20:49:45.303+05:30  INFO 16360 --- [user] [           main] org.hibernate.Version                    : HHH000412: Hibernate ORM co
2026-08-28T20:49:45.334+05:30  INFO 16360 --- [user] [           main] o.h.c.internal.RegionFactoryInitiator    : HHH000026: Second-level cac
2026-08-28T20:49:45.567+05:30  INFO 16360 --- [user] [           main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ig
2026-08-28T20:49:45.590+05:30  INFO 16360 --- [user] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2026-08-28T20:49:46.120+05:30  INFO 16360 --- [user] [           main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connec
2026-08-28T20:49:46.126+05:30  INFO 16360 --- [user] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start comple
2026-08-28T20:49:47.385+05:30  INFO 16360 --- [user] [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000489: No JTA platform
2026-08-28T20:49:47.500+05:30  INFO 16360 --- [user] [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManag
2026-08-28T20:49:47.728+05:30  WARN 16360 --- [user] [           main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is
2026-08-28T20:49:48.451+05:30  INFO 16360 --- [user] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080
2026-08-28T20:49:48.469+05:30  INFO 16360 --- [user] [           main] com.example.user.UserApplication         : Started UserApplication in
welcome to FSD
2026-08-28T21:16:59.921+05:30  WARN 16360 --- [user] [l-1 housekeeper] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Thread starv
```

Step 8: Create a new database by entering the command as ‘create database register;’ as shown below.

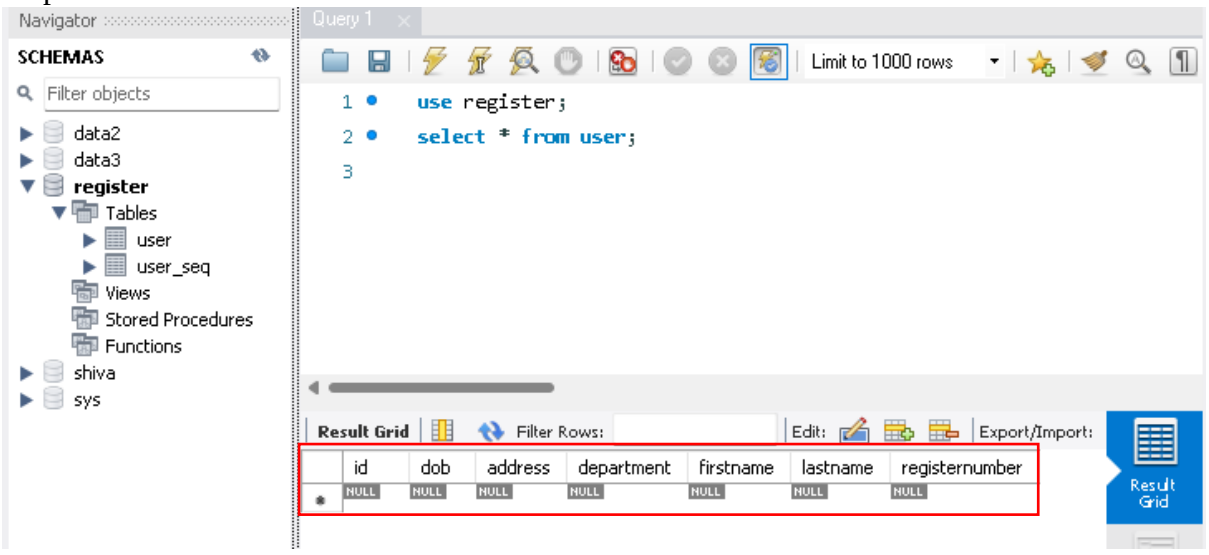


Step 9: Login to the Postman & select 'GET' method enter the command as 'localhost:8080/users' as shown below Save it Send.



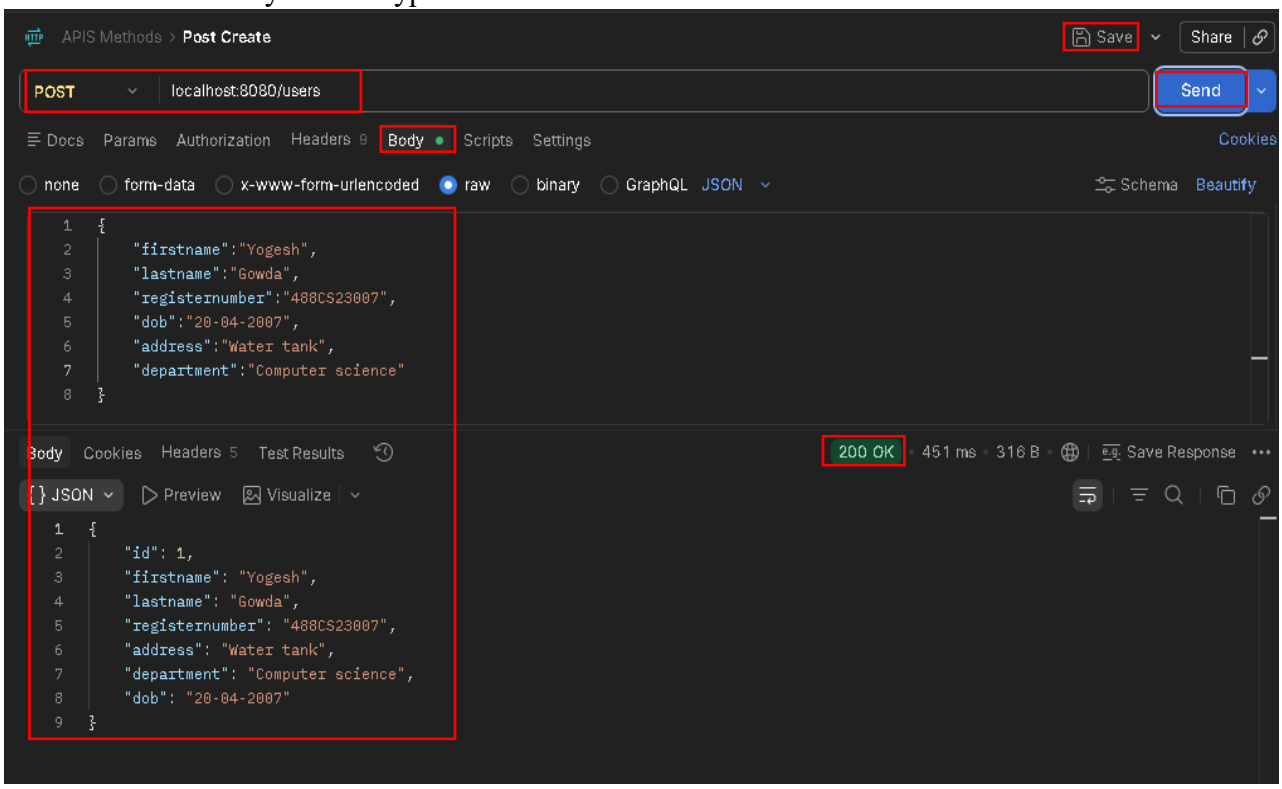
Note: Then check the response status here. If the status is 200 OK means the response has been accepted by the POSTMAN.

Step 10: Now enter the code as shown below.

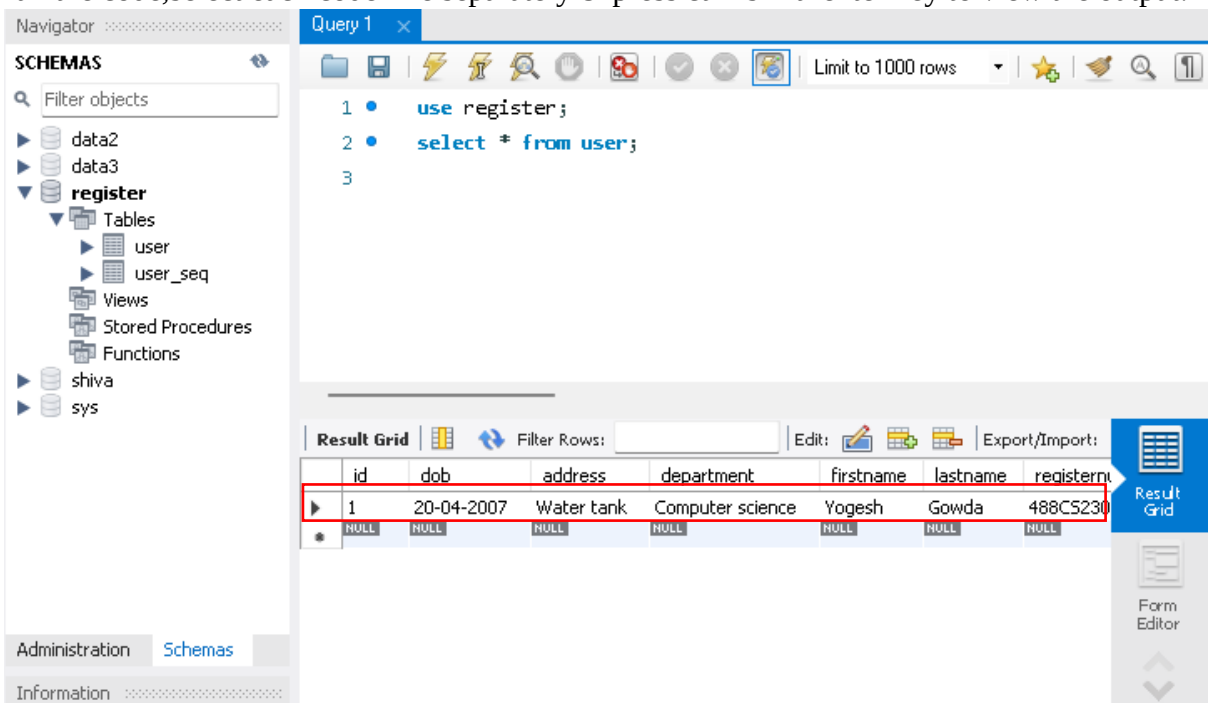


Note: To run the code, select each code line separately & press ctrl+shift+enter key to view the output.

Step 11: Select 'POST' method →enter the command as 'localhost:8080/users' as shown below → select Body → text type – JSON→ enter the below code → Save it → Send.



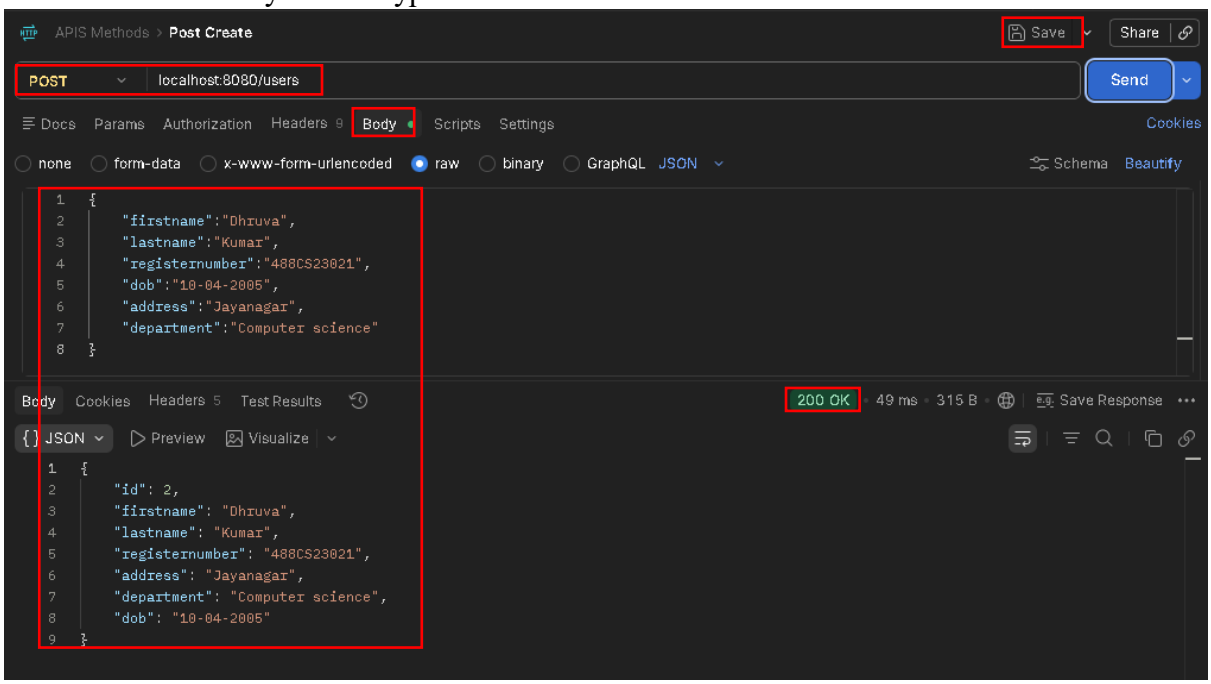
Step 12: Now go to MySQL Workbench & enter the below command as shown below. To run the code,select each code line separately & press ctrl+shift+enter key to view the output.



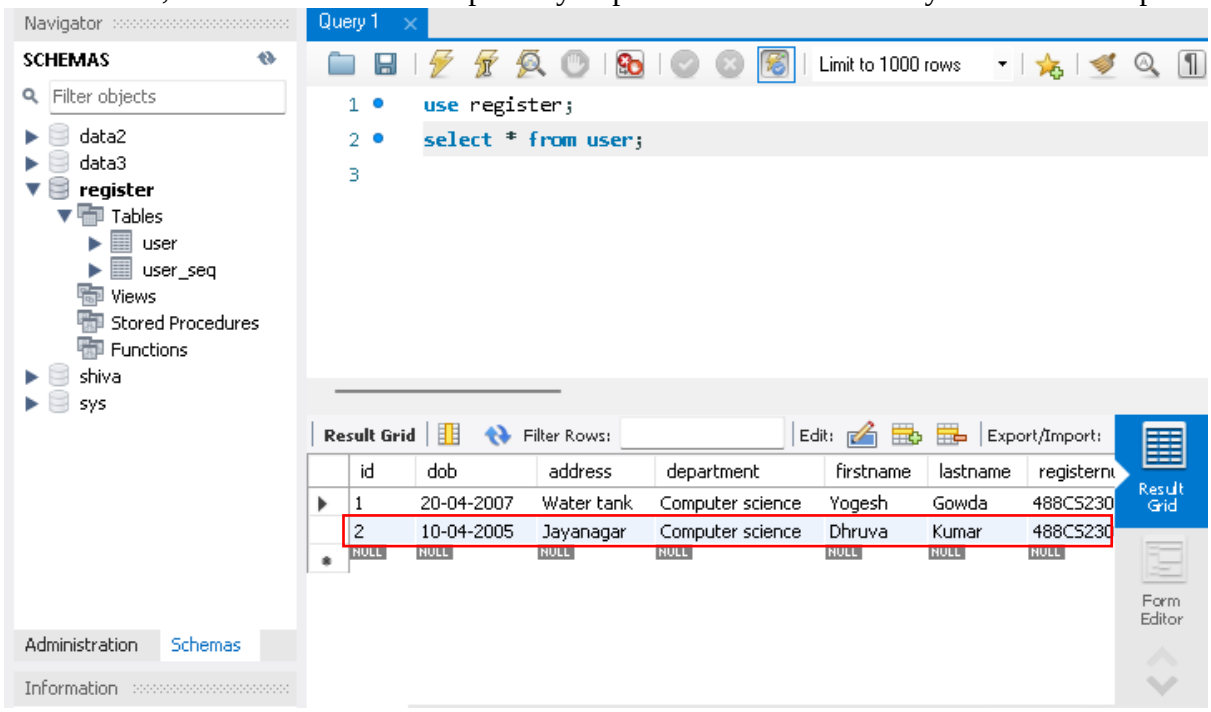
Note: Here the POST method is created successfully.

Step 13: Select 'POST' method → enter the command as 'localhost:8080/users' as shown

below → select Body → text type – JSON→ enter the below code → Save it → Send.



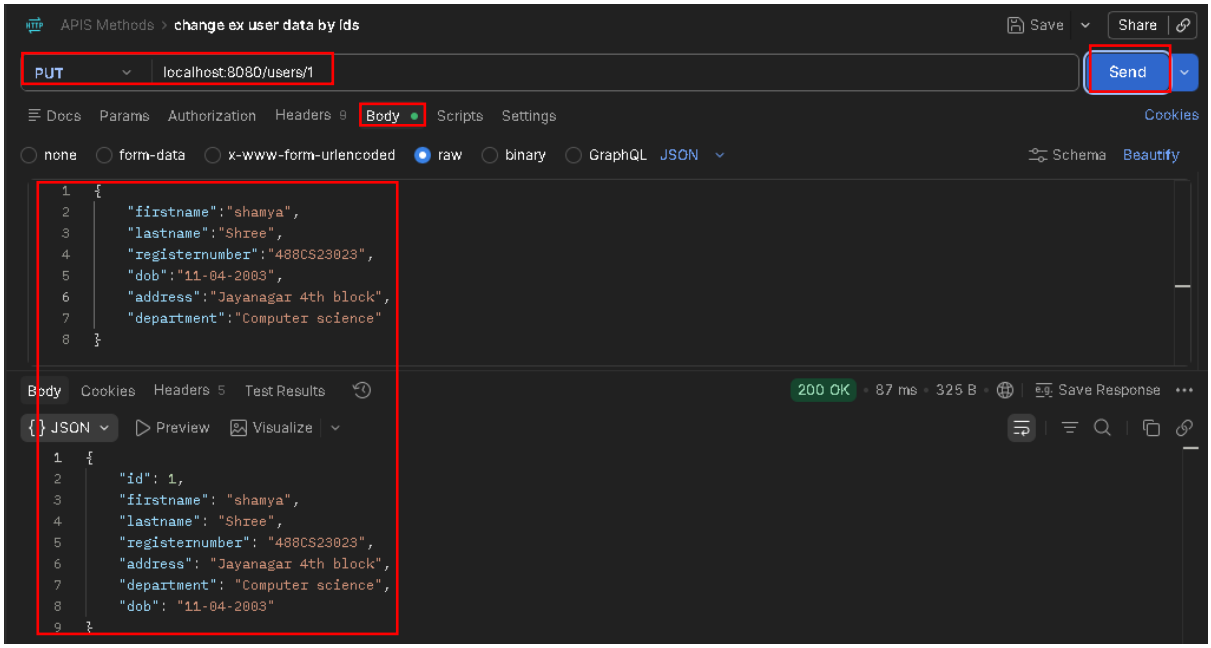
Step 14: Now go to MySQL Workbench & enter the below command as shown below. To run the code,select each code line separately & press ctrl+shift+enter key to view the output.



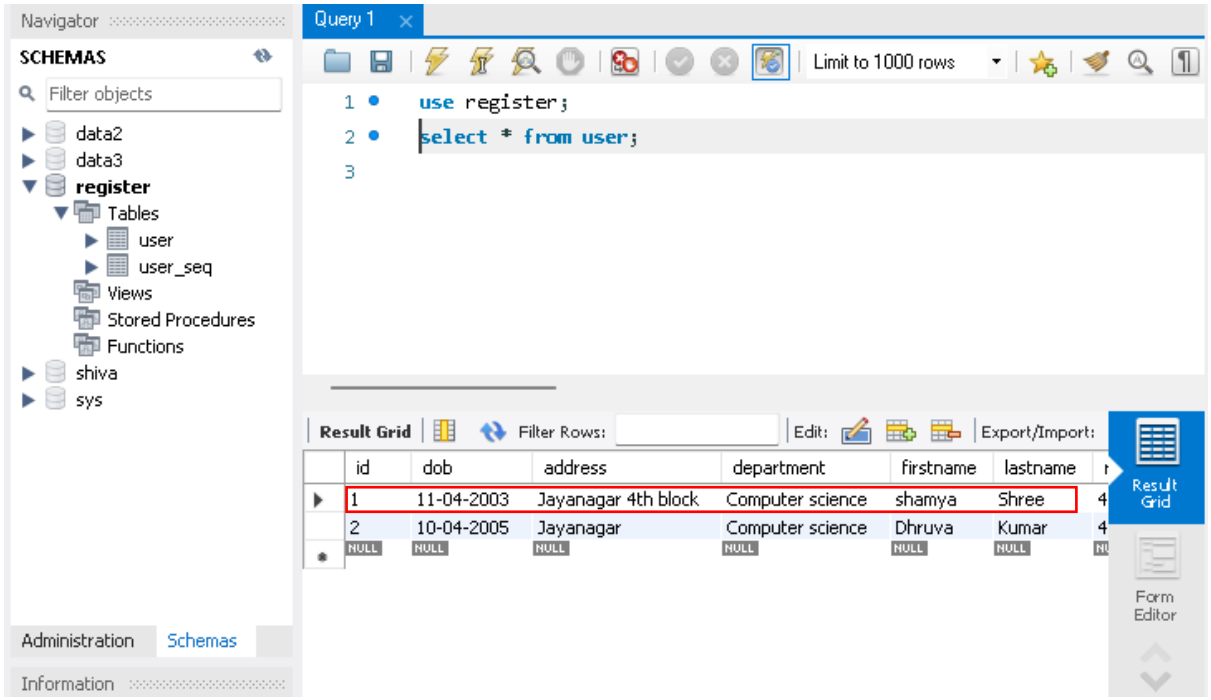
Note: Here the POST method is created successfully.

Note: Here I've created two POST methods with different ids to view the PUT method clearly.

Step 15: Select 'PUT' method → enter the command with id specified as 'localhost:8080/users/56' as shown below → select Body → text type – JSON→ enter the below code→ Save it → Send.

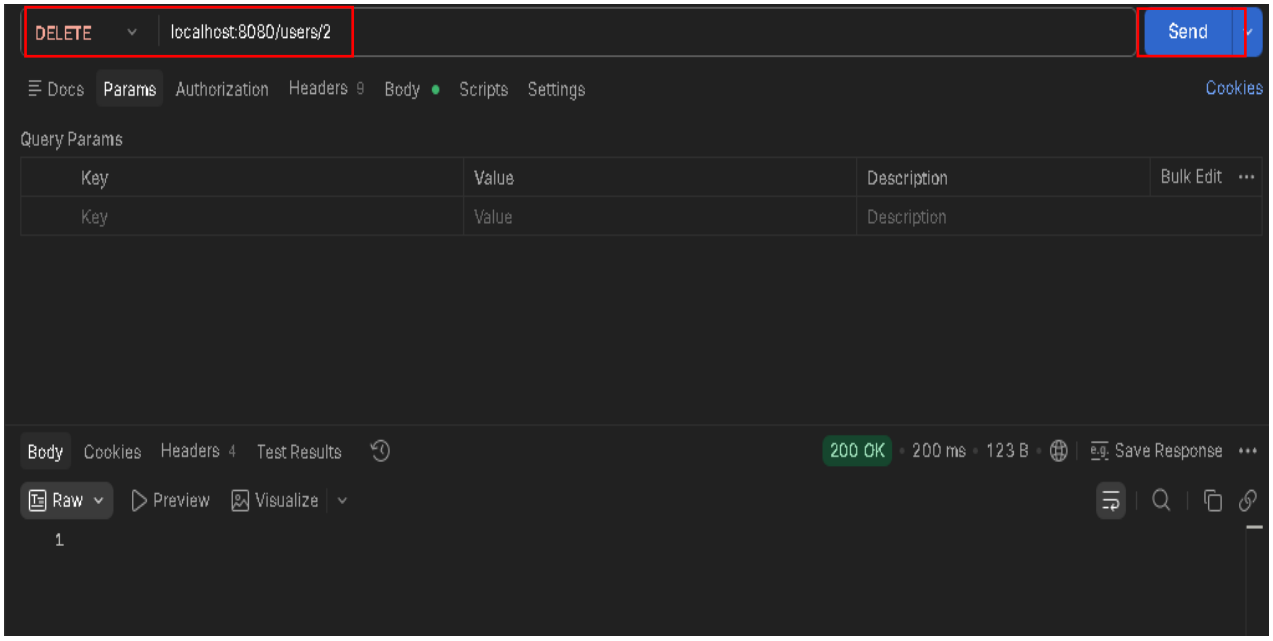


Step 16: Now go to MySQL Workbench & enter the below command as shown below. To run the code,select each code line separately & press ctrl+shift+enter key to view the output.

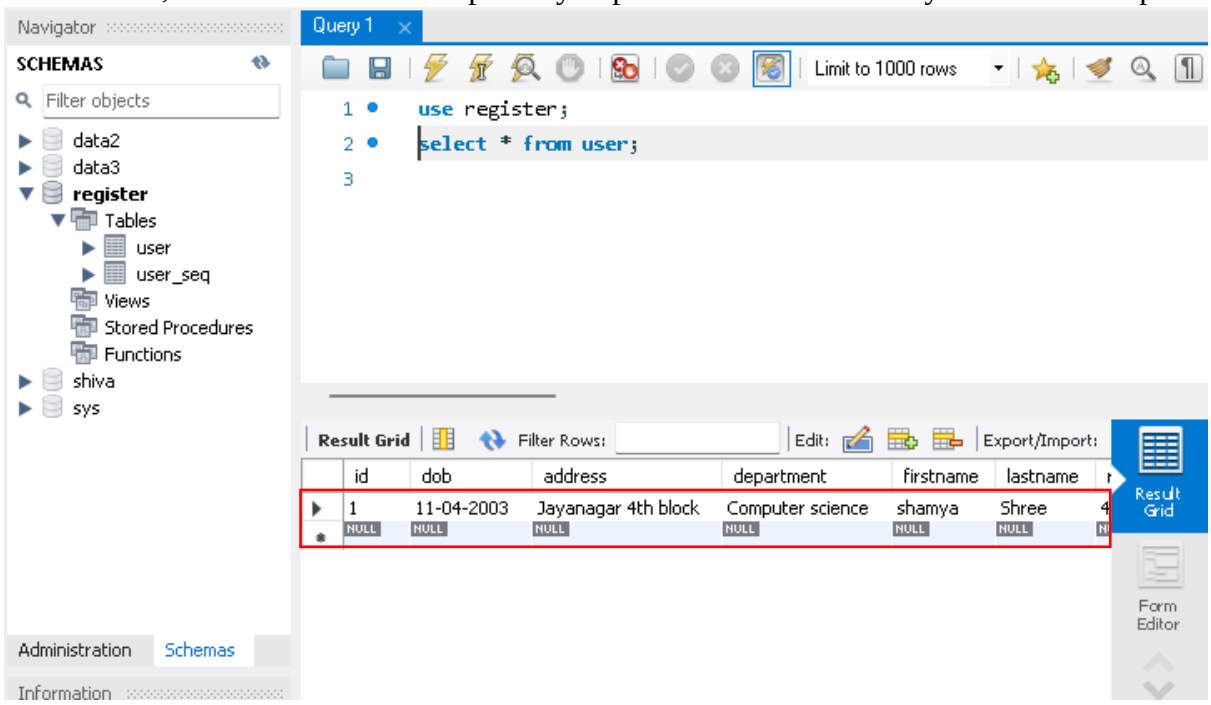


Note: Here the PUT method is updated successfully.

Step 17: Select ‘DELETE’ method → enter the command with id specified as ‘localhost:8080/users/56’as shown below → select Body → text type – JSON → enter the below code → Save it → Send.



Step 18: Now go to MySQL Workbench & enter the below command as shown below. To run the code,select each code line separately & press ctrl+shift+enter key to view the output.



Note: Here the DELETE method is deleted successfully.